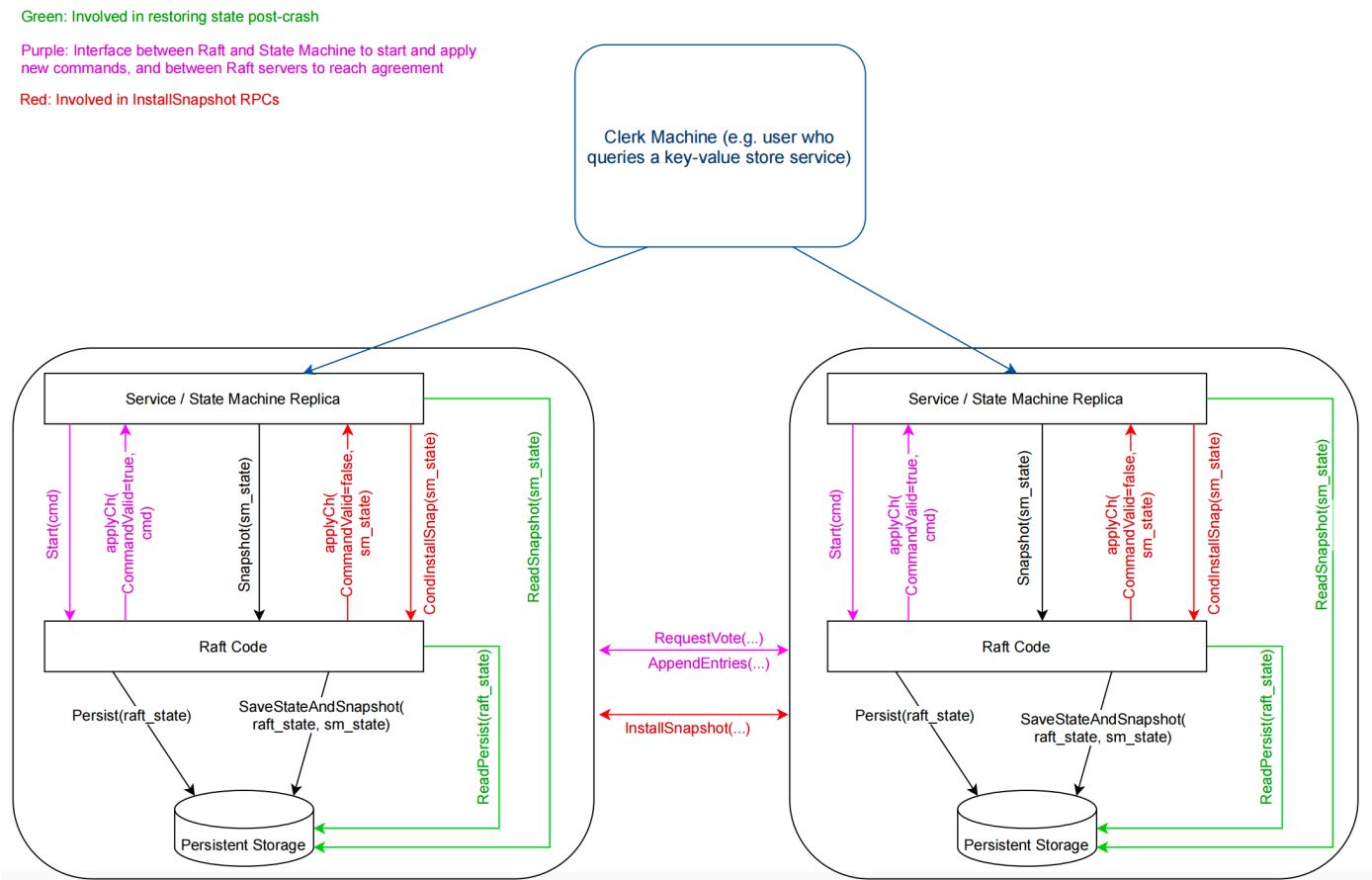


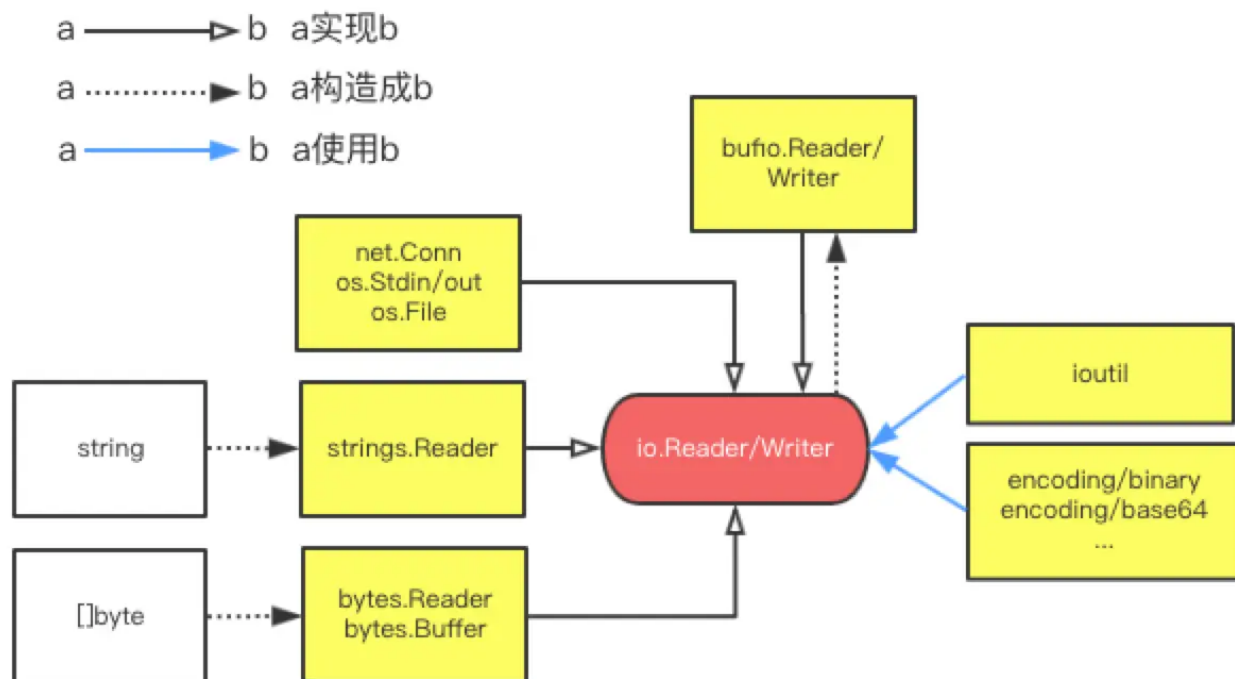
Raft 分布式实现

Raft 是一种基于复制状态机的一致性协议，它通过将上层应用程序的状态复制到多个服务器上实现容错，只要存在超过一半的服务器处于活性状态，并且互相之间能够进行通信，那么Raft所管理的服务器集群就能对外提供一致性的服务。



Raft RPC 通信数据如何序列化?

Go原生的 `pkg` 中有一些核心的 interface，其中 `io.Reader/Writer` 是比较常用的接口。很多原生的结构都围绕这个系列的接口展开，在实际的开发过程中，你会发现通过这个接口可以在多种不同的io类型之间进行过渡和转化。



CSDN @绛洞花主敏明

`encoding/gob` 是Golang包自带的一个数据结构序列化的编码/解码工具。编码使用Encoder，解码使用Decoder。`src/labgob` 对 `encoding/gob` 进行了封装，实现了Raft RPC通信数据的序列化。主要实现了 `src/labgob` 以下功能：

- 试图通过RPC发送非大写字段会产生一系列错误行为，包括神秘的错误计算和彻底崩溃。因此，在序列化和反序列的过程中Go的 `src/labgob` 的序列化器会对非大写的**字段**名称发出警告。
- 在反序列化过程工作，如果存储反序列结果的值包含非默认值，则发出警告，就像有人发送RPC但应答的数据已经被修改一样。
- 实现序列化和反序列化。序列化，将Go程序中声明的类型实例转换成可以用于网络传输的字节数组 `byte[]`。反序列化的过程与之相反。

序列化器

```
import "encoding/gob"
import "io"
import "reflect"
import "fmt"
import "sync"
import "unicode"
```

```

import "unicode/utf8"

// 序列化器
type LabEncoder struct {
    gob *gob.Encoder // src/labgob对encoding/gob实现了封装
}

// 构造方法
// io.Writer是一个Go 原生常用核心接口，很多原生的类型都实现了这个接口
// (bytes.Buffer)
func NewEncoder(w io.Writer) *LabEncoder {
    enc := &LabEncoder{}
    enc.gob = gob.NewEncoder(w)
    return enc
}

// 序列化
// 将数据e interface{}序列化到指定的容器w中(例如bytes.Buffer,底层是byte[])
func (enc *LabEncoder) Encode(e interface{}) error {
    checkValue(e)
    return enc.gob.Encode(e)
}

// 序列化
func (enc *LabEncoder) EncodeValue(value reflect.Value) error {
    checkValue(value.Interface())
    return enc.gob.EncodeValue(value)
}

```

反序列化器

```

// 序列化器
type LabDecoder struct {
    gob *gob.Decoder
}

// 构造方法

```

```
// io.Reader是一个Go 原生常用核心接口，很多原生的类型都实现了这个接口
(bytes.Buffer)
func NewDecoder(r io.Reader) *LabDecoder {
    dec := &LabDecoder{}
    dec.gob = gob.NewDecoder(r)
    return dec
}

// 反序列化器
// 将指定的容器w中(例如bytes.Buffer,底层是byte[])的数据反序列化成数据e
interface{}
func (dec *LabDecoder) Decode(e interface{}) error {
    checkValue(e)
    checkDefault(e)
    return dec.gob.Decode(e)
}

// 如果你想对一个map[string]interface{}进行编码/解码，由于map的字段是作为
interface type括起来的，那么我们需要先注册具体的类型。
func Register(value interface{}) {
    checkValue(value)
    gob.Register(value)
}

func RegisterName(name string, value interface{}) {
    checkValue(value)
    gob.RegisterName(name, value)
}
```

检测数据类型

```
var mu sync.Mutex
var errorCount int // for TestCapital
var checked map[reflect.Type]bool
```

在序列化和反序列的过程中Go的 `src/labgob` 的序列化器会对非大写的 **字段名称**发出警告。

采用反射包 `reflect` 提供的若干方法获取变量 `value interface{}` 的底层种类

```
func checkValue(value interface{}) {
    checkType(reflect.TypeOf(value))
}

func checkType(t reflect.Type) { // reflect.Type 仿射包中定义的一个接口
    k := t.Kind() // t.Kind()返回该类型的底层种类

    mu.Lock()
    // only complain once, and avoid recursion.
    if checked == nil {
        checked = map[reflect.Type]bool{}
    }
    if checked[t] {
        mu.Unlock()
        return
    }
    checked[t] = true
    mu.Unlock()

    switch k {
    case reflect.Struct:
        for i := 0; i < t.NumField(); i++ { // t.NumField()返回一个结构体类型的
            的字段数量
            f := t.Field(i) // t.Field()返回一个结构体类型第i个字段 StructFile。
            如果该类型底层类型不是Struct, 会引起程序恐慌。
            rune, _ := utf8.DecodeRuneInString(f.Name) // f.Name返回字段的名字
            string utf8.DecodeRuneInString是Go标准库unicode/utf8包中的一个函数, 它专门
            用于解码字符串中的第一个UTF-8编码的字符。
            if unicode.IsUpper(rune) == false {
                // ta da
                fmt.Printf("labgob error: lower-case field %v of %v in RPC or
                persist/snapshot will break your Raft\n",
                    f.Name, t.Name())
                mu.Lock()
                errorCount += 1
                mu.Unlock()
            }
        }
    }
}
```

```

        checkType(f.Type)
    }
    return
case reflect.Slice, reflect.Array, reflect.Ptr:
    checkType(t.Elem())
    return
case reflect.Map:
    checkType(t.Elem())
    checkType(t.Key())
    return
default:
    return
}
}

```

检测数据默认值

```

//
// 如果该值包含非默认值，则发出警告，就像有人发送RPC但应答结构体已经被修改一样。
// 如果RPC回复包含默认值，GOB将不会覆盖非默认值。
//
// warn if the value contains non-default values,
// as it would if one sent an RPC but the reply
// struct was already modified. if the RPC reply
// contains default values, GOB won't overwrite
// the non-default value.
//
func checkDefault(value interface{}) {
    if value == nil {
        return
    }
    checkDefault1(reflect.ValueOf(value), 1, "")
}

func checkDefault1(value reflect.Value, depth int, name string) {
    if depth > 3 {
        return
    }
}

```

```

t := value.Type()
k := t.Kind()

switch k {
case reflect.Struct:
    for i := 0; i < t.NumField(); i++ {
        vv := value.Field(i)
        name1 := t.Field(i).Name
        if name != "" {
            name1 = name + "." + name1
        }
        checkDefault1(vv, depth+1, name1)
    }
    return
case reflect.Ptr:
    if value.IsNil() {
        return
    }
    checkDefault1(value.Elem(), depth+1, name)
    return
case reflect.Bool,
    reflect.Int, reflect.Int8, reflect.Int16, reflect.Int32,
reflect.Int64,
    reflect.Uint, reflect.Uint8, reflect.Uint16, reflect.Uint32,
reflect.Uint64,
    reflect.Uintptr, reflect.Float32, reflect.Float64,
    reflect.String:
    if reflect.DeepEqual(reflect.Zero(t).Interface(),
value.Interface()) == false {
        mu.Lock()
        if errorCount < 1 {
            what := name
            if what == "" {
                what = t.Name()
            }
            // this warning typically arises if code re-uses the same RPC
reply

```

```

        // variable for multiple RPC calls, or if code restores
persisted
        // state into variable that already have non-default values.
        fmt.Printf("labgob warning: Decoding into a non-default
variable/field %v may not work\n",
            what)
    }
    errorCount += 1
    mu.Unlock()
}
return
}
}

```

序列化和反序列例子

```

type Result struct {
    RID int64
    Err bool
    Value string
}

func get() []byte {
    database := make(map[string]string) // kv table
    duplicate := make(map[int64]Result) // (cid, operation result), k/v
    service maintains a "duplicate table" indexed by ID

    database["0"] = "x0 1y"
    duplicate[123456789] = Result{21321, false, "x0 1y"}

    var serviceLayerState []interface{}
    serviceLayerState = append(serviceLayerState, database)
    serviceLayerState = append(serviceLayerState, duplicate)

    w := new(bytes.Buffer)
    e := NewEncoder(w)
    Register(make(map[string]string))
    Register(make(map[int64]Result))
}

```



```

    e.Encode(999)
    e.Encode(serviceLayerState)
    data := w.Bytes()
    return data
}

func main() {

    data := get()

    // w1 := bytes.NewBuffer(data)
    // e1 := NewEncoder(w1)
    // e1.Encode([]int{2,200,6})
    // e1.Encode(999*2)
    // e1.Encode(4*2)
    // data1 := w1.Bytes()

    r := bytes.NewBuffer(data)
    d := NewDecoder(r)
    var num int
    var serviceLayerState []interface{}
    if d.Decode(&num) != nil || d.Decode(&serviceLayerState) != nil {
        fmt.Println("Decode Error")
    } else {
        fmt.Println("Decode Success, %v, %v", serviceLayerState[0].
(map[string]string), serviceLayerState[1].(map[int64]Result))
    }
}

```

Raft 实例之间如何通信?

Raft 实例能够使用 `src/labrpc` 包实现RPC通信。 `src/labrpc` 能够模拟现实场景中一些网络故障，例如 PRC 延迟，RPC 消息丢失，RPC 消息乱序。其次，Raft实例之间的通信只能通过RPC调用，不能使用 Go 共享变量等技术。

使用 `src/labgob` 包来编码RPC发送的参数，能够保证参数中不包含任何程序对象的引用，即实现RPC传输数据的序列化和反序列化。

```
//  
// channel-based RPC, for 6.5840 labs.  
//  
// simulates a network that can lose requests, lose replies,  
// delay messages, and entirely disconnect particular hosts.  
//  
//  
// sends labgob-encoded values to ensure that RPCs  
// don't include references to program objects.  
//
```

labrpc 是基于 `channel` 的 `RPC`。**labrpc** 仿真了一个不可靠的网络，在该网络中可能会出现丢失请求、丢失响应、信息延迟以及与主机失去连接。**labrpc** 支持高并发RPC请求。**labrpc** 采用锁和原子操作来保证共享数据的并发安全。下图定义了RPC的 **Client** 和 **Server**。

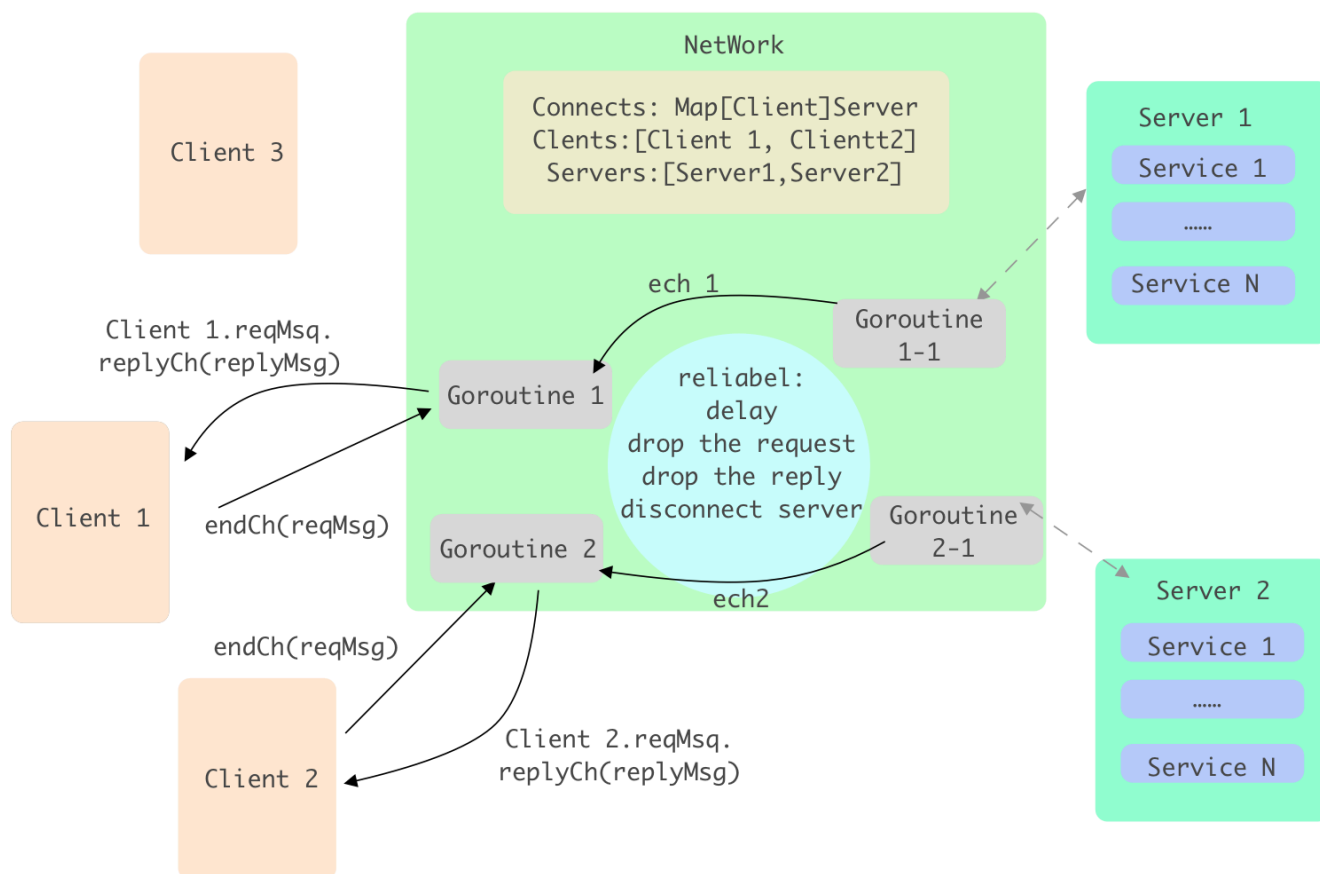
labrpc 中包含几个主要的实体：**reqMsg**, **replyMsg**, **ClientEnd**, **Server**, **Service**, **NetWork**。

- **reqMsg** 是 **Client** 发送给 **Server** 的请求信息的格式。实际的数据在 **Network** 中是以字节的形式传递，由 `labgob` 库实现将具体的数据 **Data** 转换成字节，存储在**reqMsg**。
- **replyMsg** 是 **Server** 发送给 **Client** 的响应数据的格式。实际的数据在 **Network** 中是以字节的形式传递，由 `labgob` 库实现将具体的数据 **Data** 转换成字节，存储在**replyMsg**。
- **ClientEnd** 是 `RPC` 请求的发起者，它会将数据 **Data** 通过 `labgob` 库打包字节，存储在 **reqMsg** 中，通过 **endCh** 通道发送到 **Network** 中
- **Server** 负责处理 `RPC`接受到的请求，比将 **replyMsg** 返回到 **Network** 中。
- **Service** 是 **Server** 能够为 **Client** 提供的服务，由 `reflect` 反射包实现，它会被注册到 **Server**。

- **Network** 主要提供了一个可以配置的网络环境，实现了丢失请求、丢失响应、信息延迟以及与主机失去连接。**Network** 还管理了 **Server** 与 **Client** 之间的关联关系。

注：所有的 **Client** 都会与 **Network** 共享一个 **Channel** 进行通信，即 **endCh**。

RPC 由客户端 **ClientEnd** 和服务端 **Server** 构成，**Network** 模拟了RPC调用和响应过程中的网络环境。客户端 **ClientEnd** 会通过通管 **endCh** 将RPC请求信息 **reqMsg** 发送到 **NetWork**。**NetWork** 会依据 **reqMsg** 中的信息找到对应的服务端 **Server**，调用服务端的部署的服务 **Service**，然后将服务的返回值封装成响应信息 **replyMsg** 通过通道 **replyCh** 返回给客户端 **Client**。



reqMsg 和 replyMsg

Client 和 **Server** 发送的具体数据信息都会通过 **labgob** 库编码为字节数据，然后存储在 **reqMsg.args** 或者 **replyMsg.reply** 中。

请求消息 `reqMsg` :

- `endtime` 是客户端的端口文件名
- `reqMsg.svcMeth` 指定了 Client 通过RPC需要调用的服务，例如：`Raft.AppendEntries`
- `reqMsg.args` 是具体的消息体
- `reqMsg.argsType` 指定了 Client 所发送数据 `args` 的反射类型 `reflect.Type`，方便 `Server` 获取到数据后解码。
- `reqMs` 中的 `Channel` 用于接受来自 `Server` 的响应信息 `replyMsg`，每次发送请求都会创建一个通道 `Channel`，监听来自RPC Server端的回复。

响应消息 `replyMsg` :

- `ok` 是一个bool值表示消息是否响应成功
- `reply` 表示 RPC Server 响应的数据

```
type reqMsg struct {
    endname interface{} // name of sending ClientEnd
    svcMeth string       // e.g. "Raft.AppendEntries"
    argsType reflect.Type
    args      []byte
    replyCh   chan replyMsg
}

type replyMsg struct {
    ok    bool
    reply []byte
}
```

为什么在网络传输的数据 **Data** 都需要编码为字节数据，使用时再进行解码？

因为 `labrpc` 是基于 `channel` 的 `RPC`。数据 `Data` 很多时候是 `Slice`、`Map`、`Struct` 等引用类型。引用类型的数据通过 `Channel` 从 `Client(Server)` 传递到 `Server(Client)`，很容易被 `Server(Client)` 意外修改。

RPC 客户端 ClientEnd

ClientEnd

- `Client` 维护了自己的端口文件名 `endname`（类似于 RPC 中的 `address:port`），在这是一个唯一的随机字符串，用于标识客户端。
- 同时，它还维护一个用于传递 `reqMsg` 的 `ch` 通道，该channel是NetWork持有的全局唯一的 `channel` 赋值，在实例化的过程中由 `NetWork` 传入。
- 此外，`Client` 还维护了一个名为 `done` 的通道，该channel也是由NetWork持有的全局唯一的 `channel` 赋值，接受 `Network` 关闭网络的通知，它会将自己注销掉。

```
type ClientEnd struct {
    endname interface{} // this end-point's name
    ch      chan reqMsg // copy of Network.endCh
    done    chan struct{} // closed when Network is cleaned up
}
```

`ClientEnd` 实现了 `Call` 方法，用于发送请求，并等待回复。该调用是同步调用。

- `ClientEnd` 会实例化一个请求消息实例 `reqMsg`，将需要调用的方法名和具体数据写入消息实例 `reqMsg` 中。
- 使用反射包获取该 `args interface{}` 变量的底层数据类型，并将该变量转换成 `[]byte`

```
// send an RPC, wait for the reply.
// the return value indicates success; false means that
// no reply was received from the server.
func (e *ClientEnd) Call(svcMeth string, args interface{}, reply
interface{}) bool {
    req := reqMsg{} // 封装用于发送到Network中的RPC请求
    req.endname = e.endname // 客户端的端口文件名
    req.svcMeth = svcMeth // 远程过程调用的方法名字
    req.argsType = reflect.TypeOf(args) // 参数的底层数据类型
```

```

req.replyCh = make(chan replyMsg) // 用于接受该请求返回的响应信息的通道

// 序列化请求中的数据
qb := new(bytes.Buffer)
qe := labgob.NewEncoder(qb)
if err := qe.Encode(args); err != nil {
    panic(err)
}
req.args = qb.Bytes()

//
// send the request. 通过ClientEnd能连接到Network的通道ch, 将请求req发送到Network中
//
select {
case e.ch <- req:
    // the request has been sent.
case <-e.done:
    // entire Network has been destroyed.
    return false
}

//
// wait for the reply.
//
rep := <-req.replyCh // 阻塞等到RPC请求的响应, 同步RPC调用

if rep.ok {
    // 反序列化响应体中的数据rep.reply, 将其赋值给reply
    rb := bytes.NewBuffer(rep.reply)
    rd := labgob.NewDecoder(rb)
    if err := rd.Decode(reply); err != nil {
        log.Fatalf("ClientEnd.Call(): decode reply: %v\n", err)
    }
    return true
} else { // 没有接受到服务端的响应
    return false
}
}

```

new 函数介绍

在 Go 语言中，new 函数用于动态地分配内存，返回一个指向新分配的零值的指针。它的语法如下：

```
func new (Type) *Type
```

其中，Type 表示要分配的内存的类型，new 函数返回一个指向 Type 类型的新分配的零值的指针。但是需要注意的是，new 函数只分配内存，并返回指向新分配的零值的指针，而不会初始化该内存。

bytes.Buffer 类型的用途主要是作为字节序列的缓冲区。在内部，bytes.Buffer 类型使用字节切片作为内容容器，并有一个int类型的字段作为已读计数，这个已读计数无法通过 bytes.Buffer 提供的方法计算出来。

```
type Buffer struct {  
    buf      []byte // contents are the bytes buf[off : len(buf)]  
    off      int    // read at &buf[off], write at &buf[len(buf)]  
    lastRead readOp // last read operation, so that Unread* can work  
    correctly.  
}
```

RPC 服务端 Server

服务器是服务的集合，所有服务共享相同的RPC dispatcher。例如，Raft和k/v服务器可以监听相同的rpc端点。

```
// a server is a collection of services, all sharing  
// the same rpc dispatcher. so that e.g. both a Raft  
// and a k/v server can listen to the same rpc endpoint.  
type Server struct {
```

```

mu            sync.Mutex
services map[string]*Service
count        int // incoming RPCs // RPC 调用数量
}

func (rs *Server) GetCount() int {
    rs.mu.Lock()
    defer rs.mu.Unlock()
    return rs.count
}

```

`services map[string]*Service` 是该服务端能够为远程过程调用提供的服务。

```

// 构造器
func MakeServer() *Server {
    rs := &Server{}
    rs.services = map[string]*Service{}
    return rs
}

// 向服务端增加一个新的服务Service
func (rs *Server) AddService(svc *Service) {
    rs.mu.Lock()
    defer rs.mu.Unlock()
    rs.services[svc.name] = svc
}

```

服务端 `Server` 如何根据请求参数 `reqMsg` 中的参数，找到对应的服务？

```

func (rs *Server) dispatch(req reqMsg) replyMsg {
    rs.mu.Lock() // 可能存在多个客户端并发地调用服务，对共享数据进行加锁

    rs.count += 1 // RPC调用数增加1

    // split Raft.AppendEntries into service and method
    // 字符串分割, Raft.AppendEntries => Raft AppendEntries
    dot := strings.LastIndex(req.svcMeth, ".") // find the last index of
    substr from origin str
    serviceName := req.svcMeth[:dot] // 变量类型的名字

```



```

methodName := req.svcMeth[dot+1:] // 类型携带的方法

service, ok := rs.services[serviceName] // 从服务端提供的服务中找到客户端需要调用的服务

rs.mu.Unlock()

if ok {
    return service.dispatch(methodName, req) // 调用服务端对应的服务
} else { // 没找到对应的服务
    choices := []string{}
    for k, _ := range rs.services {
        choices = append(choices, k)
    }
    log.Fatalf("labrpc.Server.dispatch(): unknown service %v in %v.%v; expecting one of %v\n",
        serviceName, serviceName, methodName, choices)
    return replyMsg{false, nil} // 响应体
}
}

```

服务 Service

一个 **server** 可以提供很多服务 **Service**。**Service** 结构体通过反射的方式为所有的外部服务提供了统一的描述方式，使得不同的外部服务可以很方便地被注册到 **server** 中。

```

// an object with methods that can be called via RPC.
// a single server may have more than one Service.
// Service 通过反射的方式重新定义了「能够通过RPC调用的某种类型的方法」
type Service struct {
    name      string // service name // 服务的名称
    rcvr      reflect.Value // 变量的值
    typ       reflect.Type // 变量的类型
    methods   map[string]reflect.Method // 提供的可访问变量的方法
}

```

`Service` 会记录某个外部服务的值 `reflect.Value` 和类型 `reflect.Type`，以及外部服务的接口 `map[string]reflect.Method`

`MakeService` 构造方法提供了将指定外部服务 `rcvr` 构建成 `Service`。

```
func MakeService(rcvr interface{}) *Service {
    // 实例化该服务
    svc := &Service{}
    svc.typ = reflect.TypeOf(rcvr) // 获取接口底层的数据类型，例如 *Raft指针类型
    svc.rcvr = reflect.ValueOf(rcvr) // 获取接口底层的值，例如 *Raft指针的值
    // Indirect returns the value that v points to. If v is a nil
    // pointer, Indirect returns a zero Value.
    // reflect.Indirect(svc.rcvr) 例如，*Raft指针所指的内存地址的值
    reflect.Value()
    // .Type().Name() 返回 Raft
    svc.name = reflect.Indirect(svc.rcvr).Type().Name()
    svc.methods = map[string]reflect.Method{}

    for m := 0; m < svc.typ.NumMethod(); m++ { // 例如：Raft 中所有的方法
        method := svc.typ.Method(m)
        mtype := method.Type
        mname := method.Name

        //fmt.Printf("%v pp %v ni %v lk %v 2k %v no %v\n",
        //    mname, method.PkgPath, mtype.NumIn(), mtype.In(1).Kind(),
        //    mtype.In(2).Kind(), mtype.NumOut())

        // 例如：对Raft中需要注册成服务的方法进行过滤
        // mtype.NumIn() != 3, 该方法参数数量需要等于3
        // mtype.In(2).Kind() != reflect.Ptr, 该方法最后一个参数底层种类需要时
        // 指针类型
        // mtype.NumOut() != 0, 该方法没有返回值
        // 例如：func (rf *Raft) RequestVote(args *RequestVoteArgs, reply
        // *RequestVoteReply) {}
        if method.PkgPath != "" || // capitalized?
            mtype.NumIn() != 3 ||
            //mtype.In(1).Kind() != reflect.Ptr ||
            mtype.In(2).Kind() != reflect.Ptr ||
```

```

    mtype.NumOut() != 0 {
        // the method is not suitable for a handler
        //fmt.Printf("bad method: %v\n", mname)
    } else {
        // the method looks like a handler
        svc.methods[mname] = method //
    }
}

return svc
}

```

`Service` 会在 `methods` 寻找名为 `methname` 的方法，并调用该方法

```

func (svc *Service) dispatch(methname string, req reqMsg) replyMsg {
    if method, ok := svc.methods[methname]; ok {
        // prepare space into which to read the argument.
        // the Value's type will be a pointer to req.argsType.
        // New returns a Value representing a pointer to a new zero value
        for the specified type. That is, the returned Value's Type is
        PtrTo(typ).
        args := reflect.New(req.argsType) // 返回一个指向该类型req.argsType的
        零值的指针, args: reflect.Value

        // bytes[] -> (reflect.Type, reflect.value)
        // 从请求题reqMsg反序列化方法参数
        // decode the argument.
        ab := bytes.NewBuffer(req.args)
        ad := labgob.NewDecoder(ab)
        ad.Decode(args.Interface()) // args.Interface() returns args's
        current value as an interface{}. 其底层类型是req.argsType

        // MethodHandler的reply
        // allocate space for the reply.
        replyType := method.Type.In(2) // 在添加服务时对方法的参数进行了检查，方
        法最后一个参数必须是指针，所以replyType是指针类型
        replyType = replyType.Elem() // 获取指针指向具体元素的类型
        replyv := reflect.New(replyType)
    }
}

```

```

// call the method.
function := method.Func
// args.Elem() 返回指针所指向的值, Elem()返回类型reflect.Value
function.Call([]reflect.Value{svc.rcvr, args.Elem(), replyv}) //
调用

// (reflect.Type, reflect.value) -> bytes[]
// 调用MethodHandler后的reply序列化
// encode the reply.
rb := new(bytes.Buffer)
re := labgob.NewEncoder(rb)
re.EncodeValue(replyv)

return replyMsg{true, rb.Bytes()}
} else { // 没找到对应的服务
    choices := []string{}
    for k, _ := range svc.methods {
        choices = append(choices, k)
    }
    log.Fatalf("labrpc.Service.dispatch(): unknown method %v in %v;
expecting one of %v\n",
        methname, req.svcMeth, choices)
    return replyMsg{false, nil}
}
}

```

网络 Network

`Network` 是 `src/labrpc` 这个库中最重要的一个组件。它提供了一个可以配置的网络环境, 实现了丢失请求、丢失响应、信息延迟以及与主机失去连接。`Network` 还管理了 `Server` 与 `Client` 之间的关联关系。

```

type Network struct {
    mu          sync.Mutex

```

```

    reliable      bool
    longDelays    bool           // pause a long time on
send on disabled connection
    longReordering bool           // sometimes delay
replies a long time
    ends          map[interface{}]*ClientEnd // ends, by name
    enabled       map[interface{}]*bool      // by end name
    servers       map[interface{}]*Server    // servers, by name
    connections   map[interface{}]*interface{} // endname ->
servername
    endCh         chan reqMsg
    done          chan struct{} // closed when Network is cleaned up
    count         int32         // total RPC count, for statistics
    bytes         int64         // total bytes send, for statistics
}

```

其中，`reliable`，`longDelay`，`longReordering` 分别定义类网络的是否可靠，网络延迟以及消息响应的延迟时间。

- `ends` 是 `ClientEnd` 在 `Network` 中的注册表，管理了网络中所有的 `(endname,*ClientEnd)`。
- `servers` 是 `Server` 在 `Network` 中的注册表，管理了网络中所有 `(servername,* Server)`。
- `enable` 标记了某些客户端 `ClientEnd` 设备是在当前网络中是可以用的。
- `connections` 定义了注册表中 `ClientEnd` 与 `Server` 两着的连接关系，类似于RPC中Client存根与Server存根的映射关系。
- `endCh` 定义了 `ClientEnd` 与 `Network` 之间唯一的信息传递的通道。

此外，`Network` 也记录中网络中总的RPC的请求连接数量 `count` 和向网络中传输的总的字节数 `bytes`

`Network` 构造方法，启动一个 `goroutine` 监听来自 `ClientEnd` 发送的请求。

```

func MakeNetwork() *Network {
    // 初始化Network数据结构

```

```

rn := &Network{}
rn.reliable = true
rn.ends = map[interface{}]*ClientEnd{}
rn.enabled = map[interface{}]bool{}
rn.servers = map[interface{}]*Server{}
rn.connections = map[interface{}](interface{}){}
rn.endCh = make(chan reqMsg)
rn.done = make(chan struct{})

// single goroutine to handle all ClientEnd.Call()s
// 启动一个Go程处理ClientEnd向网络中发送的请求reqMsg
go func() {
    for {
        select {
            case xreq := <-rn.endCh:
                // atomic包可以实现对单个变量的原子操作
                atomic.AddInt32(&rn.count, 1) // 统计发送到Network中RPC请求数
                atomic.AddInt64(&rn.bytes, int64(len(xreq.args))) // 统计网络中
数据传输量
                go rn.processReq(xreq) // 启动一个新的Go程处理该请求
            case <-rn.done: // Network已关闭
                return
        }
    }
}()

return rn
}

```

Network 提供了许多的外部接口和内置方法

```

// 关闭Network(模拟网络下线), 同时通知Client网络连接已经关闭,
ClientEnd.Call()方法将不会发送RPC请求
func (rn *Network) Cleanup() {
    close(rn.done)
}

```

设置网络可用性状态。Network 每接收到一个 Client 的 RPC 请求，就会分配一个 Goroutine 去处理请求。因此对于共享数据的访问必须上锁。

```
// yes为true表示网络是可用的
func (rn *Network) Reliable(yes bool) {
    rn.mu.Lock()
    defer rn.mu.Unlock()

    rn.reliable = yes
}

// yes为true表示网络中的rpc请求可以乱序到达服务器端Server
func (rn *Network) LongReordering(yes bool) {
    rn.mu.Lock()
    defer rn.mu.Unlock()

    rn.longReordering = yes
}

// yes为true表示网络中的请求和响应会存在长延迟
func (rn *Network) LongDelays(yes bool) {
    rn.mu.Lock()
    defer rn.mu.Unlock()

    rn.longDelays = yes
}
```

获取网络中的流量信息（RPC 请求数量和向网络中传输的字节数），对共享变量的访问使用了原子操作。

```
// get a server's count of incoming RPCs.
func (rn *Network) GetCount(servername interface{}) int {
    rn.mu.Lock()
    defer rn.mu.Unlock()

    svr := rn.servers[servername]
    return svr.GetCount()
}
```

```

func (rn *Network) GetTotalCount() int {
    x := atomic.LoadInt32(&rn.count)
    return int(x)
}

func (rn *Network) GetTotalBytes() int64 {
    x := atomic.LoadInt64(&rn.bytes)
    return x
}

```

依据客户端的端口文件名获取与客户端 `ClientEnd` 相关的信息 (包括该客户端是否启用 `enabled`, `ClientEnd` 所连接的服务端 `Server` 的名字 `servername`, 以及服务端对象 `server *Server` 和当前网络的状态 `reliable`, `longreordering`)

注意: goroutines对这些信息的访问可能是并发地, 为了保证并发安全, 需要对这些信息进行加锁。

```

func (rn *Network) readEndnameInfo(endname interface{}) (enabled bool,
    servername interface{}, server *Server, reliable bool,
    longreordering bool,
) {
    rn.mu.Lock()
    defer rn.mu.Unlock()

    enabled = rn.enabled[endname]
    servername = rn.connections[endname]
    if servername != nil {
        server = rn.servers[servername]
    }
    reliable = rn.reliable
    longreordering = rn.longReordering
    return
}

```

如何检查一个服务端 `Server` 是否能够提供服务? 当连接服务端 `Server` 的客户端 `ClientEnd` 未启用 或者 不存在该服务端, 意味着服务端 `Server` 宕机了


```

func (rn *Network) isServerDead(endname interface{}, servername
interface{}, server *Server) bool {
    rn.mu.Lock()
    defer rn.mu.Unlock()

    if rn.enabled[endname] == false || rn.servers[servername] != server
{
        return true
    }
    return false
}

```

`ClientEnd` 将请求发送到 `Network` 中，请求后续会怎么样？`Network` 会为为每一个请求 `reqMsg` 创建一个专门为其服务的 `goroutine`，该 `goroutine` 会执行 `processReq` 处理请求。

```

func (rn *Network) processReq(req reqMsg) {
    // 依据发送请求的客户端名称req.endname获取请求的具体信息
    enabled, servername, server, reliable, longreordering :=
rn.readEndnameInfo(req.endname)

    if enabled && servername != nil && server != nil { // 确保存在服务器能
够处理该请求
        if reliable == false {
            // short delay
            ms := (rand.Int() % 27)
            // 使得处理请求goroutine短暂睡眠一段时间模拟网络延迟
            time.Sleep(time.Duration(ms) * time.Millisecond) // type
Duration int64
        }

        if reliable == false && (rand.Int()%1000) < 100 { // 在网络不可用的
情况下，模拟存在10%的概率，请求会在网络中丢失
            // drop the request, return as if timeout
            req.replyCh <- replyMsg{false, nil}
            return
        }
    }
}

```

```

// execute the request (call the RPC handler).
// in a separate thread so that we can periodically check
// if the server has been killed and the RPC should get a
// failure reply.
ech := make(chan replyMsg)
go func() { // 在主Goroutine内创建一个子Coroutine 调用服务端对应的RPC
handler处理请求reqMsg
    r := server.dispatch(req) // 调用服务端对应的RPC handler处理请求
reqMsg
    ech <- r
}()

// 主Coroutine与子Goroutine通过通道ech进行通信

// wait for handler to return,
// but stop waiting if DeleteServer() has been called,
// and return an error.
var reply replyMsg
replyOK := false
serverDead := false
for replyOK == false && serverDead == false {
    select {
    case reply = <-ech: // 主goroutine阻塞式等待子goroutine的响应
        replyOK = true
    case <-time.After(100 * time.Millisecond):
        // 启动定时器, 如果100ms后还没有收到来自子Gortinue的响应, 则查询服务端
是否宕机了
        serverDead = rn.isServerDead(req.endname, servername, server)
        if serverDead { // 如果服务端宕机, 则replyOK = false, serverDead
= true, 主goroutine结束阻塞

            // 如果一个任务不是定时任务, 那么该任务一定要被终止
            go func() { //注意: 这里怎么实现的目的是, 使用一个额外的go程将ech中
的数据取出, 使得之前创建的goroutine能够被销毁
                <-ech // drain channel to let the goroutine created
earlier terminate
            }()
        }
    }
}

```

```

}

// do not reply if DeleteServer() has been called, i.e.
// the server has been killed. this is needed to avoid
// situation in which a client gets a positive reply
// to an Append, but the server persisted the update
// into the old Persister. config.go is careful to call
// DeleteServer() before superseding the Persister.
serverDead = rn.isServerDead(req.endname, servername, server)

if replyOK == false || serverDead == true {
    // server was killed while we were waiting; return error.
    req.replyCh <- replyMsg{false, nil}
} else if reliable == false && (rand.Int()%1000) < 100 { // 网络不可靠, 模拟在RPC响应消息时, 存在10%的概率丢失响应体
    // drop the reply, return as if timeout
    req.replyCh <- replyMsg{false, nil}
} else if longreordering == true && rand.Intn(900) < 600 { // 模拟存在网络延迟时, 有30%的概率会发生响应延迟
    // delay the response for a while
    ms := 200 + rand.Intn(1+rand.Intn(2000)) // 随机响应延迟时间
    // Russ points out that this timer arrangement will decrease
    // the number of goroutines, so that the race
    // detector is less likely to get upset.
    // 重点
    time.AfterFunc(time.Duration(ms)*time.Millisecond, func() { //
time.AfterFunc启动一个任务, 在指定ms时间后, 执行匿名函数中定义的任务体
        atomic.AddInt64(&rn.bytes, int64(len(reply.reply)))
        req.replyCh <- reply
    })
} else { // 直接响应该消息
    atomic.AddInt64(&rn.bytes, int64(len(reply.reply)))
    req.replyCh <- reply
}
} else { // 无法找到能够处理该请求服务端程序
    // simulate no reply and eventual timeout.
    ms := 0
    if rn.longDelays {
        // let Raft tests check that leader doesn't send

```

```

    // RPCs synchronously.
    ms = (rand.Int() % 7000)
} else {
    // many kv tests require the client to try each
    // server in fairly rapid succession.
    ms = (rand.Int() % 100)
}
time.AfterFunc(time.Duration(ms)*time.Millisecond, func() {
    req.replyCh <- replyMsg{false, nil}
})
}
}

```

`Network` 提供可以构建客户端 `ClientEnd` 的方法，其会构造一个 `ClientEnd` 并将其注册到网络 `Network` 中。

```

// create a client end-point.
// start the thread that listens and delivers.
func (rn *Network) MakeEnd(endname interface{}) *ClientEnd {
    rn.mu.Lock()
    defer rn.mu.Unlock()

    if _, ok := rn.ends[endname]; ok {
        log.Fatalf("MakeEnd: %v already exists\n", endname)
    }

    // 构造客户端实例
    e := &ClientEnd{}
    e.endname = endname
    e.ch = rn.endCh
    e.done = rn.done
    // 将客户端注册到网络Network中
    rn.ends[endname] = e
    rn.enabled[endname] = false
    rn.connections[endname] = nil

    return e
}

```

`AddService` 将一个指定的 `Server` 注册到网络 `Network` 中。 `DeleteServer` 将指定的 `Server` 从网络中删除。

```
func (rn *Network) AddServer(servername interface{}, rs *Server) {
    rn.mu.Lock()
    defer rn.mu.Unlock()

    rn.servers[servername] = rs
}

func (rn *Network) DeleteServer(servername interface{}) {
    rn.mu.Lock()
    defer rn.mu.Unlock()

    rn.servers[servername] = nil
}
```

将一个 `ClientEnd` 与 `Server` 连接

```
// connect a ClientEnd to a server.
// a ClientEnd can only be connected once in its lifetime.
func (rn *Network) Connect(endname interface{}, servername
interface{}) {
    rn.mu.Lock()
    defer rn.mu.Unlock()

    rn.connections[endname] = servername
}
```

启用和禁用一个 `ClientEnd`

```
// enable/disable a ClientEnd.
func (rn *Network) Enable(endname interface{}, enabled bool) {
    rn.mu.Lock()
    defer rn.mu.Unlock()

    rn.enabled[endname] = enabled
}
```

如何模拟分布式环境?

全局配置信息

在testing包中包含一下结构体:

testing.T: 这就是我们平常使用的单元测试

testing.F: 模糊测试, 可以自动生成测试用例

testing.B: 基准测试. 对函数的运行时间进行统计.

testing.M: 测试的钩子函数, 可预置测试前后的操作.

testing.PB: 测试时并行执行.

Go 对单元测试函数要求如下:

1. 文件名形如: `xxx_test.go`
2. 函数签名形如: `func TestXxx(t *testing.T)`

使用如下命令运行测试用例(test.run 指定运行某一个函数):

```
go test -test.run TestFuncName
```

`src/raft` 包中的 `config.go` 定义了Raft算法运行时所处的模拟的分布式环境。该 `config` 文件定义整个系统模拟运行时的相关配置信息。

```
type config struct {
    mu          sync.Mutex
    t           *testing.T // 单元测试包
    finished    int32
    net         *labrpc.Network // 网络
    n           int
    rafts       []*Raft // raft 算法
    applyErr    []string // from apply channel readers
    connected   []bool   // whether each server is on the net
    saved       []*Persister
```

```

    endnames    [][]string           // the port file names each sends
to
    logs        []map[int]interface{} // copy of each server's committed
entries
    lastApplied []int
    start       time.Time // time at which make_config() was called
// begin()/end() statistics
    t0         time.Time // time at which test_test.go called cfg.begin()
    rpcs0       int        // rpcTotal() at start of test
    cmds0       int        // number of agreements
    bytes0      int64
    maxIndex    int
    maxIndex0   int
}

```

其中，`testing.T` 是Go语言内置的单元测试结构体类型

- `net *labrpc.Network` Raft 算法运行的网络，模拟网络。
- `rafts []*Raft` 所有的 Raft 状态机
- `connect []bool`: `connect[i]` 表示name为 `i` 的Raft 状态机是否在网络中
- `saved []*Persister`: `saved[i]` 表示name为 `i` 的Raft 状态机用于持久化状态机信息和日志信息的持久化器，用来模拟本地磁盘。
- `endnames [][]string`: 每一Raft状态机都持久一个唯一能访问其他Raft状态机服务的客户端 `ClientEnd`，因此一个Raft状态机可以持有多个 `ClientEnd`。每一Raft状态机有唯一的提供服务的 `Server`。 `endnames` 二维数组定义 Raft 状态机之间的连接映射关系。 `endnames` 可以看作是一个 `n x n` 的邻接矩阵。
- `n` 表示Raft状态机的数量
- `logs []map[int]interface{}`: `logs[i]` 表示Raft状态机 `i` 已经提交的日志记录的拷贝，这个 `logs[i]` 就相当于Raft的上层应用服务的内存状态。
- `lastApplied []int`: `lastApplied[i]` 记录Raft状态机 `i` 上一次提交的日志的编号

`config` 结构体构造器，构造器参数：

- `n` 表示Raft状态机的数量
- `unreliable` 表示网络是是否是可靠的
- `snapshot` 表示是否启用记录快照

```
var ncpu_once sync.Once

func make_config(t *testing.T, n int, unreliable bool, snapshot bool)
*config {
    utils.InitLogger() // init logger

    ncpu_once.Do(func() { // 原子操作，只会执行一次
        if runtime.NumCPU() < 2 { // 返回当前系统的 CPU 核数量
            fmt.Printf("warning: only one CPU, which may conceal locking
bugs\n")
        }
        rand.Seed(makeSeed()) // 设置随机种子
    })
    runtime.GOMAXPROCS(4) // 设置最大的可同时使用的 CPU 核数

    cfg := &config{}
    cfg.t = t
    cfg.net = labrpc.MakeNetwork()
    cfg.n = n // 有n个server
    cfg.applyErr = make([]string, cfg.n)
    cfg.rafts = make([]*Raft, cfg.n)
    cfg.connected = make([]bool, cfg.n)
    cfg.saved = make([]*Persister, cfg.n)
    cfg.endnames = make([][]string, cfg.n)
    cfg.logs = make([]map[int]interface{}, cfg.n)
    cfg.lastApplied = make([]int, cfg.n)
    cfg.start = time.Now() // 当前本地时间

    cfg.setunreliable(unreliable) // 配置网络的可靠性， 网络不可靠可能会导致请
求丢失或者响应丢失

    cfg.net.LongDelays(true) // 网络延迟

    applier := cfg.applier
```



```

if snapshot { // 记录快照
    applier = cfg.applierSnap
}
// create a full set of Rafts.
for i := 0; i < cfg.n; i++ {
    cfg.logs[i] = map[int]interface{}{} // 分配内存空间
    cfg.start1(i, applier) // 启动server 1
}

// connect everyone
for i := 0; i < cfg.n; i++ {
    cfg.connect(i) // server 及其相关的client端 接入net
}

return cfg
}

```

如何模拟启动或重启一个Raft状态机?

`config` 提供了能够启动或重启Raft状态机的方法 `start1()` 。

1. 将Raft持有的 `ClientEnd` 注册到 `Network` 中

- 在启动或重启一个Raft状态机之前，首先会将该Raft状态机关闭 `cfg.crash1(i)`
- 为了方便该Raft状态机与 `Network` 中的其余Raft状态机通信，该Raft状态机需要创建 `n` 个客户端 `ClientEnd`，每个客户端 `ClientEnd` 具有不同的端口文件名 `endname`，每个 `ClientEnd` 负责连接一个Raft状态机的服务端 `Server`。
- 也就是说Raft状态机持有这些客户端 `ClientEnd`，能够与其余Raft状态机通信

2. 快速恢复

- 使用该Raft实例的持久化器 `Persister` 从持久化的快照中恢复Raft状态机的状态信息。如果是重启一个Raft状态机，则从最近的快照中恢复状态信息。

3. 将 `Raft Server` 注册到 `Network` 中

- 使用 `src/raft` 模块中提供的构造方法 `Make()` 创建Raft实例，每一个Raft实例会持有一个 `applyCh := make(chan ApplyMsg)`。每个Raft实例可以通过 `applyCh` 将需要提交的日志信息传输 `config`，这个过程是模拟Leader提交日志记录，将日志记录中的命令应用到状态机上。
- `config` 环境会启动一个提交器 `applier` 的 `goroutine` 监听来自该Raft实例的提交请求：`go applier(i, applyCh)`。
- 每一个Raft实例还会持有一个持久化器 `Persister` 模拟将日志和快照持久化存储。
- 使用 `src/labrpc` 中的 `MakeServer()` 构造 `Raft Server`，使用 `MakeService()` 构造Raft提供的服务 `Service`，将服务 `Service` 注入到 `Server` 中
- 将 `Raft Server` `i` 注册到 `Network` 中，`i` 是 `Raft Server` 的名字

```
// start or re-start a Raft.
// if one already exists, "kill" it first.
// allocate new outgoing port file names, and a new
// state persister, to isolate previous instance of
// this server. since we cannot really kill it.
func (cfg *config) start1(i int, applier func(int, chan ApplyMsg)) {
    cfg.crash1(i)

    // a fresh set of outgoing ClientEnd names.
    // so that old crashed instance's ClientEnds can't send.
    cfg.endnames[i] = make([]string, cfg.n) // 为该Raft分配端口文件名
    for j := 0; j < cfg.n; j++ {
        cfg.endnames[i][j] = randstring(20)
    }

    // a fresh set of ClientEnds.
    ends := make([]*labrpc.ClientEnd, cfg.n) // 为Raft创建n个不同的
    ClientEnd, 并将每一个ClientEnd与其余的Raft的Server建立网络连接
    for j := 0; j < cfg.n; j++ {
        // 创建一个ClientEnd并将其注册到Network中
        ends[j] = cfg.net.MakeEnd(cfg.endnames[i][j]) // create an client
        end named cfg.endnames[i][j]
    }
}
```

```

    cfg.net.Connect(cfg.endnames[i][j], j) // cfg.endnames[i][j]是
ClientEnd的名称, j是Server的名称
}

cfg.mu.Lock()

cfg.lastApplied[i] = -1 // 0

// a fresh persister, so old instance doesn't overwrite
// new instance's persisted state.
// but copy old persister's content so that we always
// pass Make() the last persisted state.
if cfg.saved[i] != nil { // 如果重启一个Raft, 重快照中加载其状态机的状态信
息
    cfg.saved[i] = cfg.saved[i].Copy()

    snapshot := cfg.saved[i].ReadSnapshot()
    if snapshot != nil && len(snapshot) > 0 {
        // mimic KV server and process snapshot now.
        // ideally Raft should send it up on applyCh...
        err := cfg.ingestSnap(i, snapshot, -1)
        if err != "" {
            cfg.t.Fatal(err)
        }
    }
} else { // 如果是启动一个Raft, 那么就初始化快照持久化器
    cfg.saved[i] = MakePersister()
}

cfg.mu.Unlock()

applyCh := make(chan ApplyMsg)

rf := Make(ends, i, cfg.saved[i], applyCh) // 创建Raft实例

cfg.mu.Lock()
cfg.rafts[i] = rf
cfg.mu.Unlock()

```

```

go applier(i, applyCh)

// 构造Raft Server, Raft提供的服务Service, 将服务Service注入到Server中,
svc := labrpc.MakeService(rf)
srv := labrpc.MakeServer()
srv.AddService(svc)
cfg.net.AddServer(i, srv) // 将Raft Server注册到Network中, i 是Raft
Server的名字
}

```

Raft 状态机的网络连接

除了构造方法外，`config` 还提供了其他的接口，例如将一个Raft状态机 `i` 接入网络Network中的方法 `connect(i)`

- 将Raft 状态机 `i` 的服务器连接状态置为 `true`
- 启用该Raft状态机 `i` 与其余Raft状态机 `j` 通信所需要的 `ClientEnd`
- 启用其余Raft状态机 `j` 与该Raft状态机 `i` 通信所需要的 `ClientEnd`

类似地，使得Raft 状态机 `i` 从网络Network中下线的方法 `disconnect(i)`。

- 将Raft 状态机 `i` 的服务器连接状态置为 `false`
- 禁用该Raft状态机 `i` 与其余Raft状态机 `j` 通信所需要的 `ClientEnd`
- 禁用其余Raft状态机 `j` 与该Raft状态机 `i` 通信所需要的 `ClientEnd`

```

// attach server i to the net.
func (cfg *config) connect(i int) {
    // fmt.Printf("connect(%d)\n", i)

    cfg.connected[i] = true // Raft server i 是在net上

    // outgoing ClientEnds
    for j := 0; j < cfg.n; j++ { // 启用该Raft状态机i 与其余Raft状态机j通信
        所需的ClientEnd
        if cfg.connected[j] {
            endname := cfg.endnames[i][j]
            cfg.net.Enable(endname, true)
        }
    }
}

```

```

    }
}

// incoming ClientEnds
for j := 0; j < cfg.n; j++ { // 启用其余Raft状态机j与该Raft状态机i通信所需要的ClientEnd
    if cfg.connected[j] {
        endname := cfg.endnames[j][i]
        cfg.net.Enable(endname, true)
    }
}
}

// detach server i from the net.
func (cfg *config) disconnect(i int) {
    // fmt.Printf("disconnect(%d)\n", i)

    cfg.connected[i] = false // 将Raft server i 从在net上删除

    // outgoing ClientEnds
    for j := 0; j < cfg.n; j++ { // 禁用该Raft状态机i 与其余Raft状态机j通信所需要的ClientEnd
        if cfg.endnames[i] != nil {
            endname := cfg.endnames[i][j]
            cfg.net.Enable(endname, false)
        }
    }

    // incoming ClientEnds
    for j := 0; j < cfg.n; j++ { // 禁用其余Raft状态机j与该Raft状态机i通信所需要的ClientEnd
        if cfg.endnames[j] != nil {
            endname := cfg.endnames[j][i]
            cfg.net.Enable(endname, false)
        }
    }
}
}

```

如何模拟关闭一个Raft 状态机?

- 使得Raft 状态机 `i` 从网络Network中下线的方法 `disconnect(i)` , 会禁用 Network注册表中所有的Raft 状态机 `i` 持有的 `ClientEnd`
- 从 `Network` 的注册表中删除已注册的 `Raft Server` 。
- 销毁Raft 状态机实例 `rf.Kill()`
- 持久化日志和快照信息, 这里是「模拟持久化过程」

```
// shut down a Raft server but save its persistent state.
func (cfg *config) crash1(i int) {
    cfg.disconnect(i)
    cfg.net.DeleteServer(i) // disable client connections to the server.

    cfg.mu.Lock()
    defer cfg.mu.Unlock()

    // a fresh persister, in case old instance
    // continues to update the Persister.
    // but copy old persister's content so that we always
    // pass Make() the last persisted state.
    if cfg.saved[i] != nil {
        cfg.saved[i] = cfg.saved[i].Copy()
    }

    rf := cfg.rafts[i]
    if rf != nil {
        cfg.mu.Unlock()
        rf.Kill()
        cfg.mu.Lock()
        cfg.rafts[i] = nil
    }

    if cfg.saved[i] != nil {
        raftlog := cfg.saved[i].ReadRaftState()
        snapshot := cfg.saved[i].ReadSnapshot()
    }
}
```

```

    cfg.saved[i] = &Persister{}
    cfg.saved[i].Save(raftlog, snapshot)
}
}

```

模拟不支持Snapshot的日志提交

Raft 实例通过其自身持有的 `applyCh chan ApplyMsg` 的通道，向通道 `applyCh` 中发生一个提交请求。`config` 的提交器 `applier` 会监听该提交请求并处理请求

```

// applier reads message from apply ch and checks that they match the
log
// contents
func (cfg *config) applier(i int, applyCh chan ApplyMsg) {
    for m := range applyCh {
        if m.CommandValid == false {
            // ignore other types of ApplyMsg
        } else {
            cfg.mu.Lock()
            err_msg, prevok := cfg.checkLogs(i, m)
            cfg.mu.Unlock()
            if m.CommandIndex > 1 && prevok == false {
                err_msg = fmt.Sprintf("server %v apply out of order %v", i,
m.CommandIndex)
            }
            if err_msg != "" {
                log.Fatalf("apply error: %v", err_msg)
                cfg.applyErr[i] = err_msg
                // keep reading after error so that Raft doesn't block
                // holding locks...
            }
        }
    }
}
}

```

Raft 一致性协议应该保证所有状态机的已提交日志是一样的，这样以相同的顺序执行日志记录中的命令后，状态机的状态才能保持一致。

因此，`checkLogs` 方法首先会检查所有状态机已提交的日志记录中，索引为 `m.CommitIndex` 的日志记录中的命令与参数 `m.Command` 的命令是否一致。如果不一致那么所实现的Raft程序代码可能存在Bug。

最后，依据 `(i int, m ApplyMsg)` 更新 `cfg.logs[i]` 中的日志记录

```
func (cfg *config) checkLogs(i int, m ApplyMsg) (string, bool) {
    err_msg := ""
    v := m.Command
    for j := 0; j < len(cfg.logs); j++ {
        if old, oldok := cfg.logs[j][m.CommandIndex]; oldok && old != v {
            log.Printf("%v: log %v; server %v\n", i, cfg.logs[i],
cfg.logs[j])
            // some server has already committed a different value for this
entry!
            err_msg = fmt.Sprintf("commit index=%v server=%v %v != server=%v
%v",
                m.CommandIndex, i, m.Command, j, old)
        }
    }
    _, prevok := cfg.logs[i][m.CommandIndex-1]
    cfg.logs[i][m.CommandIndex] = v
    if m.CommandIndex > cfg.maxIndex {
        cfg.maxIndex = m.CommandIndex
    }
    return err_msg, prevok
}
```

模拟支持Snapshot的日志提交

`ingestSnap` 方法可以实现通过快照 `snapshot []byte`，重建Raft状态机 `i` 的已提交日志 `c.logs[i]`。

- 将快照字节数组 `snapshot []byte` 反序列化成 `lastIncludedIndex` 和 `xlog []interface{}`
- 依据快照中的数据 `xlog []interface{}` 更新 `cfg.logs[i]`
- 更新 `cfg.lastApplied[i] = lastIncludedIndex`
- 方法的参数列表中的参数 `index` 主要是用于 校验 序列化的数据是否正确

```
// returns "" or error string
func (cfg *config) ingestSnap(i int, snapshot []byte, index int)
string {
    if snapshot == nil {
        log.Fatalf("nil snapshot")
        return "nil snapshot"
    }
    r := bytes.NewBuffer(snapshot)
    d := labgob.NewDecoder(r)
    var lastIncludedIndex int
    var xlog []interface{}
    if d.Decode(&lastIncludedIndex) != nil ||
        d.Decode(&xlog) != nil {
        log.Fatalf("snapshot decode error")
        return "snapshot Decode() error"
    }
    if index != -1 && index != lastIncludedIndex {
        err := fmt.Sprintf("server %v snapshot doesn't match
m.SnapshotIndex", i)
        return err
    }
    cfg.logs[i] = map[int]interface{}{}
    for j := 0; j < len(xlog); j++ {
        cfg.logs[i][j] = xlog[j]
    }
    cfg.lastApplied[i] = lastIncludedIndex
    return ""
}
```

`applierSnap` 是上层应用服务提供支持快照的日志提交。类似地，Raft 实例通过其自身持有的 `applyCh chan ApplyMsg` 的通道，向通道 `applyCh` 中发生一个提交请求。`config` 的提交器 `applierSnap` 会监听该提交请求并处理请求。`config` 会在启动一个Raft状态机时，启动一个后台协程 `applierSnap` 监听提交请求。

- 当 `m.SnapshotValid=true` 时，表示 `config` 收到来自Raft 状态机的提交请求，该提交请求是使用快照 `m.Snapshot` 重建状态机已提交日志
- 当 `m.CommandValid=true` 时，表示 `config` 收到来自Raft状态机的普通命令提交请求，`config` 执行普通命令提交器。每当有 `SnapShotInterval` 命令提交后，上层应用服务都会调用Raft的接口 `rf.Snapshot(m.CommandIndex, w.Bytes())` 执行一次快照持久化，将已提交日志存储到快照中。

```
// periodically snapshot raft state
func (cfg *config) applierSnap(i int, applyCh chan ApplyMsg) {
    cfg.mu.Lock()
    rf := cfg.rafts[i]
    cfg.mu.Unlock()
    if rf == nil {
        return // ???
    }

    for m := range applyCh {
        err_msg := ""
        if m.SnapshotValid {
            cfg.mu.Lock()
            err_msg = cfg.ingestSnap(i, m.Snapshot, m.SnapshotIndex)
            cfg.mu.Unlock()
        } else if m.CommandValid {
            if m.CommandIndex != cfg.lastApplied[i]+1 {
                err_msg = fmt.Sprintf("server %v apply out of order, expected index %v, got %v", i, cfg.lastApplied[i]+1, m.CommandIndex)
            }

            if err_msg == "" {
                cfg.mu.Lock()
                var prevok bool
                err_msg, prevok = cfg.checkLogs(i, m)
            }
        }
    }
}
```

```

    cfg.mu.Unlock()
    if m.CommandIndex > 1 && prevok == false {
        err_msg = fmt.Sprintf("server %v apply out of order %v", i,
m.CommandIndex)
    }
}

cfg.mu.Lock()
cfg.lastApplied[i] = m.CommandIndex
cfg.mu.Unlock()

if (m.CommandIndex+1)%SnapShotInterval == 0 {
    w := new(bytes.Buffer)
    e := labgob.NewEncoder(w)
    e.Encode(m.CommandIndex)
    var xlog []interface{}
    for j := 0; j <= m.CommandIndex; j++ {
        xlog = append(xlog, cfg.logs[i][j])
    }
    e.Encode(xlog)
    rf.Snapshot(m.CommandIndex, w.Bytes())
}
} else {
    // Ignore other types of ApplyMsg.
}
if err_msg != "" {
    log.Fatalf("apply error: %v", err_msg)
    cfg.applyErr[i] = err_msg
    // keep reading after error so that Raft doesn't block
    // holding locks...
}
}
}

```

Raft 一致性协议

Raft 数据结构

日志条目 `LogEntry`，日志条目包括待执行的命令 `Command` 和Leader创建该日志条目时的任期 `Term`

```
type LogEntry struct {  
    Command interface{}  
    Term int  
}
```

在Raft 算法执行的过程中，其维护了一些数据信息：

- `mu sync.Mutex` **互斥锁**用于对 **slice、map** 等线程不安全的数据访问进行加锁，保证多线程对共享数据的访问和修改是独占的。在一个 Go 协程调用 `Lock` 方法获得锁后，其他请求锁的协程都会阻塞在 `Lock` 方法，直到锁被释放。
- `peers []*labrpc.ClientEnd` 当前Raft状态机持的 `ClientEnd`，Raft状态机能够通过 `ClientEnd` 对其余Raft状态机的远程服务 `Service` 发起调用。
- `persister *Persister`：持久化器 `Persister` 持久当前Raft实例的持久化状态。
- `me int` 是当前Raft状态机在 `peers` 中的索引，`me` 也是当前Raft状态机的名字。Raft集群中n个Raft状态机都是从0~n-1命名的
- `currentLeaderId` 表示当前Raft实例 **已知的** Raft集群中 `Leader` 的 `Id`，因此，`currentLeaderId` 不一定是最新的Leader

一个Raft状态必须维护一下信息：

- `log []LogEntry` Raft 算法主要的目标是管理日志复制，`log` 表示当前Raft状态机上的 **日志**。日志条目 `LogEntry` 包括待执行的命令 `Command` 和Leader创建该日志条目时的任期 `Term`。第一个日志条目所在的下标从 **从1开始**
- `applyCh chan ApplyMsg`：是在构造Raft实例时，由外界传入的参数。当Raft状态机提交日志时，会向 `applyCh` 发送提交命令请求。

- `timer *time.Timer` : 是一个定时器, 用于实现选举开始时 `Candidate` 的随机选举超时时间

需要持久化的信息

- `state string` : 表示当前Raft处于 `Follower`、`Candidate`、`Leader` 三种角色中的哪一种角色
- `currentTerm int` : 表示Raft实例 **已知的** 最新的任期, 初始化为0, 单调递增
- `votedFor int` : 用于记录在选举过程中, 当前的Raft 状态机为谁投了票

在所有的Raft实例上都是 **易失性** 的数据

- `commitIndex int` : 已知的被提交的所有 `LogEntry` 中最高的索引 `logIndex`, 初始化为0, 单调递增
- `lastApplied int` : 被应用到状态机中所有 `LogEntry` 中最高的索引 `logIndex`, 初始化为0, 单调递增

在角色为 `Leader` 的Raft 实例上 **易失性** 的数据, 该类数据在每次选举后重新初始化。

- `nextIndex []int` : `nextIndex[i]` 表示当前Raft状态机作为 `Leader` 将要发送给Raft 状态机 `i` 的下一个 `LogEntry` 的索引, 其初始值是当前**Leader**最后一个日志记录索引 `last log index + 1`。 `nextIndex` 主要用于日志一致性检查。
- `matchIndex []int` : `matchIndex[i]` 表示已知当前Raft状态机作为 `Leader` 复制给Raft状态机 `i` 的日志记录的最高索引值, 其初始值为0, 单调递增。 `matchIndex` 主要检查日志是否在超过一半的状态机上复制成功。

条件变量 `sync.Cond`, 用来协调想要访问共享资源的那些 `goroutine`, 当共享资源的状态发生变化的时候, 它可以用来通知被互斥锁阻塞的 `goroutine`。

- `applyCond *sync.Cond` :
- `leaderCond *sync.Cond` :
- `candidateCond *sync.Cond` :
- `triggerCond []*sync.Cond` :
- `electCond []*sync.Cond` :

用于实现选举机制：

- `wgForTicket sync.WaitGroup`：
- `ticket int32`：

用于实现快照Snapshot:

- `snapshot []byte`：**Snapshot的本地缓存**。为了提升Raft KV服务的性能，Leader需要为其他Follower同步 `snapshot` 信息时，可以直接从Raft状态机的本地缓存中读取信息，**无需访问磁盘**。这样做可以提高日志同步的性能。
- `lastIncludedIndex int`：**反序列后的Snapshot的本地缓存数据**。`lastIncludedIndex` 表示Snapshot中最后一个日志记录的索引。
- `lastIncludedTerm int`：**反序列后的Snapshot的本地缓存数据**。`lastIncludedTerm` 表示Snapshot中最后一个日志记录的任期
- `serviceLayerState []interface{}`：**反序列后的Snapshot的本地缓存数据**。上层服务程序的状态
- `snapshotCond []*sync.Cond`：

```
type Raft struct {
    mu          sync.Mutex          // Lock to protect shared access to
    this peer's state
    peers       []*labrpc.ClientEnd // RPC end points of all peers
    persister   *Persister          // Object to hold this peer's
    persisted state
    me          int                  // this peer's index into peers[]
    dead        int32                // set by Kill()

    currentLeaderId int

    // state a Raft server must maintain.
    log []LogEntry // sliceIndex, log entries
    timer *time.Timer // election timeout timer
    applyCh chan ApplyMsg // send and apply log entries to state machine
    by using it

    // Persistent state on all servers
```

```

state string // follower, candidate or leader
currentTerm int // greater than zero
votedFor int

// Volatile state on all servers
commitIndex int // logIndex, index of highest log entry known to be
committed (initialized to 0, increases monotonically)
lastApplied int // logIndex, index of highest log entry applied to
state machine (initialized to 0, increases monotonically)

// Volatile state on leaders
// Reinitialized after election
nextIndex []int // logIndex, for each server, index of the next log
entry to send to that server (initialized to leader last log index +
1)
matchIndex []int // logIndex, for each server, index of highest log
entry known to be replicated on server (initialized to 0, increases
monotonically)

// conditional variable
applyCond *sync.Cond // trigger applier
leaderCond *sync.Cond // trigger heartbeat or checker
candidateCond *sync.Cond
triggerCond []*sync.Cond // trigger request of heartbeat or request
of replication
electCond []*sync.Cond

// parallel election
wgForTicket sync.WaitGroup
ticket int32

// snapshot 本地缓存数据
snapshot []byte
// 反序列化后的snapshot本地缓存数据
lastIncludedIndex int // the snapshot replaces all entries up
through and including this index
lastIncludedTerm int // term of lastIncludedIndex
serviceLayerState []interface{}

```

```

    snapshotCond []*sync.Cond
}

```

Raft 构造器 其对外提供了构造方法。 `peers` 参数是Raft实例的持有的所有的 `ClientEnd`，用于与其他Raft实例之间的RPC通信。 `me` 参数是当前Raft实例在 `peers` 数组中的下标。 `ApplyMsg` 参数：上层应用服务希望Raft在提交每一个命令后，能够向 `applyCh` 信道中发生一个 `ApplyMsg`。

```

func Make(peers []*labrpc.ClientEnd, me int,
    // Your initialization code here (2A, 2B, 2C).
    // initialize from state persisted before a crash
    persister *Persister, applyCh chan ApplyMsg) *Raft {
    rf := &Raft{
        peers:peers, // raft's leader election requires that len(rf.peers)
        // is a odd. Then, it's effective to avoid the 'brain split'.
        persister:persister,
        me:me,
        state:FOLLOWER, // Raft状态机的初始状态是Follower
        currentTerm:0, // 0
        votedFor:-1, // -1表示还没有进行投票
        timer:time.NewTimer(timeout(configs.ELECTION_TIME_START,
        configs.ELECTION_TIME_END)), // 选举的随机超时时间
        nextIndex:make([]int, len(peers)),
        matchIndex:make([]int, len(peers)),
        log:[]LogEntry{LogEntry{nil,0}}, // log日志是从下标从1开始存放日志,
        // 因此在初始化的时候预先存放一个no-op日志记录
        applyCh:applyCh,
        lastApplied:-1, // -1
        triggerCond:make([]*sync.Cond, len(peers)),
        electCond:make([]*sync.Cond, len(peers)),
        snapshotCond:make([]*sync.Cond, len(peers)),
        ticket:1, // 1
        lastIncludedIndex:-1, // -1
        snapshot:nil, // nil

        currentLeaderId:rand.New(rand.NewSource(time.Now().UnixNano())).Intn(
        len(peers)), // random
    }
}

```



```

rf.leaderCond = sync.NewCond(&rf.mu)
rf.candidateCond = sync.NewCond(&rf.mu)
rf.applyCond = sync.NewCond(&rf.mu)

for i := 0 ; i < len(rf.peers) ; i ++ {
    rf.triggerCond[i] = sync.NewCond(&rf.mu)
    rf.electCond[i] = sync.NewCond(&rf.mu)
    rf.snapshotCond[i] = sync.NewCond(&rf.mu)
}

// raft reads persistent state and application reads snapshot
rf.readPersist(persister.ReadRaftState()) // 从持久层中读取Raft状态
rf.readSnapshot(persister.ReadSnapshot()) // 初始化Snapshot本地缓存

utils.Debugger(utils.DInfo, "S%d %s, Created", rf.me, rf.getState())

// start ticker goroutine to start elections
// daemon goroutine
// election
go rf.ticker()
if configs.HIGH_CONCURRENCY_ELECTION {
    go rf.committee()
    go rf.ticketer()
}

// replication
go rf.trigger()
go rf.heartbeat()
go rf.checker()
go rf.applier()
// snapshot
go rf.syncer()

return rf
}

```

Raft 对外方法

Start(command)

`start` 方法是Raft状态机对外暴露的接口，负责接受来自客户端的命令 `command interface{}`，能够使得Raft将一条命令追加到日志中，`Start()` 方法是非阻塞式的，会立即返回(`index, term, isLeader`)，不会等待「日志追加完成」。

- 在处理来自客户端的命令时，当前的Raft状态机会检查自己是否是集群中的 `Leader`。如果不是Leader，则拒绝客户端的请求。如果是 `Leader`，则依据命令 `command` 和当前的任期创建一条日志记录 `LogEntry`，并将该日志记录追加到日志 `log` 中
- 为了提高并发性，Raft 状态机(Leader)会创建一个单独的后台协程 `go trigger()` 处理日志复制相关逻辑。协程 `trigger()` 会为每一个其余的Raft状态机(Follower)创建一个goroutine，该goroutine负责处理Leader的日志记录复制到对应Follower状态机上。goroutine每次处理完日志复制后，都会处于阻塞状态 `rf.triggerCond[server].Wait()`，并等待唤醒。
- 当Leader 收到来自客户端的命令 `command` 后，将命令封装成 `LogEntry` 添加到 `log` 中后，就会使用Condition的广播机制唤醒所有负责处理日志复制的goroutines，`rf.triggerCond[i].Broadcast()`。

```
func (rf *Raft) Start(command interface{}) (int, int, bool) {
    rf.mu.Lock()
    defer rf.mu.Unlock()
    index, term, isLeader := -1, rf.currentTerm, rf.state == LEADER
    if !isLeader { return -1, -1, false } // this server isn't the leader
    rf.log = append(rf.log, LogEntry{command, term}) // append log entry
    index = rf.sliceIndex2logIndex(len(rf.log) - 1) //
sliceIndex2logIndex

    rf.persist()

    utils.Debugger(utils.DLeader, "S%d Leader, Start agreement, index
%d, term %d", rf.me, index, term)
    for i := 0; i < len(rf.peers); i ++ { rf.triggerCond[i].Broadcast()
}
```

```
    return index, term, isLeader
}
```

GetCurrentLeaderId()

获取当前Raft 状态机已经的LeaderId

```
func (rf *Raft) GetCurrentLeaderId() int {
    rf.mu.Lock()
    defer rf.mu.Unlock()
    return rf.currentLeaderId
}
```

GetState()

获取当前Raft状态信息，会返回当前Raft的「任期」和其是否认为自己是「Leader」

```
func (rf *Raft) GetState() (int, bool) {
    rf.mu.Lock()
    defer rf.mu.Unlock()
    return rf.currentTerm, rf.state == LEADER
}
```

ReadState()

获取当前Raft的角色状态

```
func (rf *Raft) ReadState() string {
    rf.mu.Lock()
    defer rf.mu.Unlock()
    return rf.state
}
```

Kill()

设置Raft状态机的状态为宕机状态

```
func (rf *Raft) Kill() {
    atomic.StoreInt32(&rf.dead, 1)
    // Your code here, if desired.
}
```

Raft 内置方法

Raft 还内置很多方法，这些方法不暴露给外界使用，例如 `killed()` 返回当前Raft状态机是否宕机。

`timeout(start int64, end int64)` 返回一个在区间 `[start, end]` 内的一个随机超时时间，单位为ms，返回类型为 `time.Duration`

`heartime(ms int64)` 返回一个指定的时间，单位为ms，返回类型为 `time.Duration`

```
func (rf *Raft) killed() bool {
    z := atomic.LoadInt32(&rf.dead)
    return z == 1
}

func timeout(start int64, end int64) time.Duration {
    ms := start + (rand.Int63() % (end - start)) // ms [start, end]
    return time.Duration(ms) * time.Millisecond
}

func heartime(ms int64) time.Duration {
    return time.Duration(ms) * time.Millisecond
}
```

`isStaleServer` 方法能够比较当前任期 `rf.currentTerm` 与方法参数 `term`。如果 `rf.currentTerm < term`，则当前的Raft状态机可能由于网络分区等原因脱落Raft集群：

- 更新当前任期为term
- 重新设置选举定时器时间

```
func (rf *Raft) isStaleServer(term int) bool {
    rf.mu.Lock()
    defer rf.mu.Unlock()
```

```

if rf.currentTerm < term {
    utils.Debugger(utils.DWarn, "S%d %s, A stale server, current term: %d, overdue term: %d.", rf.me, rf.state, term, rf.currentTerm)
    rf.state = FOLLOWER
    rf.currentTerm = term
    rf.timer.Reset(timeout(configs.ELECTION_TIME_START, configs.ELECTION_TIME_END))

    rf.persist() // update rf.currentTerm
    return true
}
return false
}

```

Raft 配置文件

`src/configs/configuration.go` 中定义了Raft相关的配置信息：

-

```

package configs

// utils package
const KVRAFT_DEBUG = false
const RAFT_DEBUG = false

// raft package
const MAXREQUESTCOUNT = 1
const FASTBACKUP = true
const HIGH_CONCURRENCY_ELECTION = true

const ELECTION_TIME_START = 105
const ELECTION_TIME_END = 335

const HEARTBEAT_TIME_START = 45
const HEARTBEAT_TIME_END = 46

```

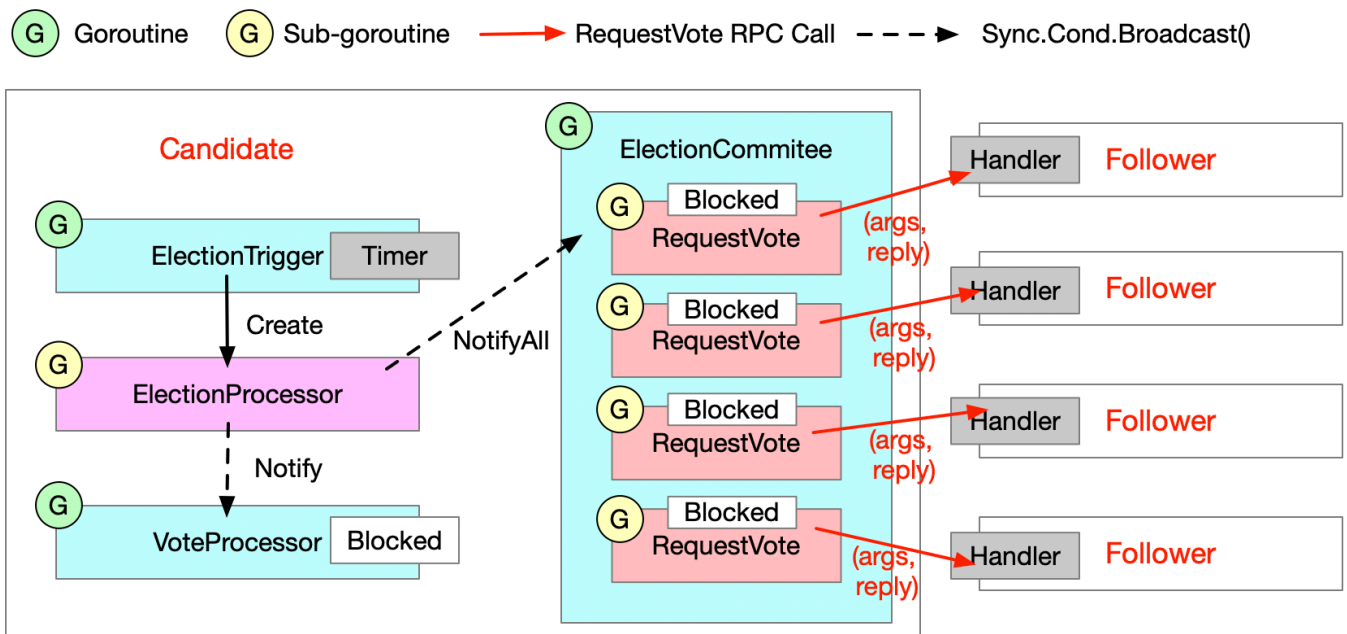
```

const REQUEST_TIMEOUT_START = 33
const REQUEST_TIMEOUT_END = 34

// kvraft package
const KVRAFT_REQUEST_TIMEOUT_START = 160
const KVRAFT_REQUEST_TIMEOUT_END = 161

```

选举机制



选举触发器 (ElectionTrigger)

在使用 **Raft 构造器** 构造一个Raft状态机后，会在后台启动一个协程 `go ticker()`。协程 `ticker()` 主要负责触发选举，当 **选举定时器**(`rf.timer`) 设置的超时时间到后，协程 `ticker()` 会先检查当前Raft状态机的状态(是否是Leader, 是否宕机)。如果当前Raft状态机满足选举条件就会触发发起一次选举：

- 发起选举及选举的过程由专门的协程 `go rf.elect()` 负责
- 协程 `ticker()` 在触发完选举后，会进入下一轮的超时等待中

```

func (rf *Raft) ticker() {
    for rf.killed() == false {
        rf.timer.Reset(timeout(configs.ELECTION_TIME_START,
configs.ELECTION_TIME_END))
        <- rf.timer.C
        if !rf.isState(LEADER) && rf.killed() == false { // followers or
candidates timeout and start election
            if configs.HIGH_CONCURRENCY_ELECTION {
                go rf.elect()
            } else {
                go rf.startElection()
            }
        }
    }
}

```

选举处理器 (ElectionProcessor)

选举触发器 `ticker()` 在发起一次选举后，会进入下一轮超时等待，而选举处理器 `elect()` 负责具体的选举。

在每次发起选举之前，需要预先处理一些数据：

- 重新初始化票数 `ticket` 为 1，因为 `Candidate` 在发起一轮新的选举后，会先给自己投一票
- 如果将每一个Raft状态机的投票过程视为一个投票任务，那么新一轮选举中待完成的投票任务为 `len(rf.peers) - 1`。使用 `sync.WaitGroup` 来使得当前协程等待多个任务的完成，`rf.wgForTicket.Add(len(rf.peers) - 1)`。
- 将当前任期数量 `currentTerm + 1`，Raft状态 `state` 及的状态转换成 `Candidate`，更新 `votedFor`。
- 重新初始化 `nextIndex` 和 `matchIndex` 数组

唤醒选举和投票相关的协程：

- `rf.candidateCond.Broadcast()` 唤醒投票处理器处理投票结果
- `rf.electCond[i].Broadcast()` 唤醒选举委员会进行投票

```

func (rf *Raft) elect() {
    rf.wgForTicket.Add(len(rf.peers) - 1)
    rf.resetTicket() // reset 重新初始化当前的票数

    rf.mu.Lock()
    rf.currentTerm += 1 // 更新任期
    rf.state = CANDIDATE // 更新Raft的角色为Candidate
    rf.votedFor = rf.me // Candidate先为自己投票
    utils.Debugger(utils.DTrace, "S%d %s, Start a new election, %dth
term", rf.me, rf.state, rf.currentTerm)

    rf.persist() // update rf.votedFor

    for i := 0; i < len(rf.peers); i ++ { // reinitialized nextIndex
        rf.nextIndex[i] = rf.sliceIndex2logIndex(len(rf.log)) //
sliceIndex2logIndex
        rf.matchIndex[i] = 0
    }
    rf.mu.Unlock()

    rf.candidateCond.Broadcast() // rf.ticketer, go live
    for i := 0; i < len(rf.peers); i ++ { rf.electCond[i].Broadcast() }
    // vote request goroutine, go live
}

```

选举委员会 (ElectionCommittee)

Raft 集群中所有的Raft状态机构成了选举委员会。委员会中的每一个成员都持有一票，可以将该票投给自己，也可以将该票投给其他成员。选举委员会是在Raft构造器在构造Raft实例后成立的，在后台启动一个协程 `go rf.committee()` 负责管理选举委员会的具相关事项。

- `committee()` 协程会为委员会中除了当前 `Candidate` 之外的每一个成员创建一个单独的 `goroutine`。该 `goroutine` 负责处理当前 `Candidate` 与其他成员之间的投票相关的RPC请求和响应
- 默认情况下，`goroutine`处于阻塞状态 `rf.electCond[server].Wait()`，它会被选举处理器唤醒。


```

func (rf *Raft) committee() {
    for i := 0; i < len(rf.peers); i ++ {
        if rf.me == i { continue }

        go func(rf *Raft, server int) {
            for rf.killed() == false {
                rf.mu.Lock()
                rf.electCond[server].Wait() // block

                var lastLogIndex int
                var lastLogTerm int
                if len(rf.log) == 0 {
                    lastLogIndex, lastLogTerm = rf.lastIncludedIndex,
rf.lastIncludedTerm
                } else {
                    lastLogIndex, lastLogTerm =
rf.sliceIndex2logIndex(len(rf.log) - 1), rf.log[len(rf.log) - 1].Term
                }

                args := &RequestVoteArgs{ // 封装请求的参数
                    Term:rf.currentTerm,
                    CandidateId:rf.me,
                    LastLogIndex:lastLogIndex,
                    LastLogTerm:lastLogTerm,
                }
                reply := &RequestVoteReply{} // 封装响应的参数
                rf.mu.Unlock()

                ch := make(chan bool)

                if !rf.isState(CANDIDATE) { // 如果当前的Candidate没有赢得选举，而
是其他的Candidate赢得选取
                    rf.wgForTicket.Done() // 此Goroutine负责的请求投票的任务结束
                    continue
                }

                go func(rf *Raft, server int, ch chan bool, args
*RequestVoteArgs, reply *RequestVoteReply) { // go request
                    ch <- rf.sendRequestVote(server, args, reply) // 发送请求

```

```

}(rf, server, ch, args, reply)

select
{
case ok := <- ch: // 阻塞等待
    if(!ok) { // 超过最大请求次数, 结束此次请求投票任务
        rf.wgForTicket.Done()
        continue
    }
    // 阻塞等待一定会结束, 如果由于网络不可靠, 网络延迟高, 就会结束此次任
    务

    case <-time.After(timeout(configs.REQUEST_TIMEOUT_START,
configs.REQUEST_TIMEOUT_END)): // long delay, timeout, timer
        rf.wgForTicket.Done()
        go func(ch chan bool) { // get the value from ch so that
this goroutine(sync-call-goroutine) can be destroyed
            <- ch
        }(ch)
        continue
    }

    // 成功得到响应信息, 比较当前任期与接受者返回的任期
    if rf.isStaleServer(reply.Term) { // check term, stale server
        rf.wgForTicket.Done() // 如果当前Raft是一个落单者, 则结束此次请求
        投票任务

        continue
    }

    utils.Debugger(utils.DVote, "S%d Candidate, Receive reply from
S%d, %dth term, %v", rf.me, server, rf.getCurrentTerm(),
reply.VoteGranted)

    if reply.VoteGranted { rf.addTicket() } // 如果接受者给该
Candidate投票, 则投票计数器+1
    rf.wgForTicket.Done()
}
}(rf, i)
}
}

```

投票处理器 (VoteProcessor)

投票处理器 `ticketer()` 也是在Raft构造器在构造Raft实例后，在后台启动的协程。投票处理器 `ticketer()` 只负责对投票的结果判决，具体的投票过程由委员会中的各个Raft状态机负责。

因此，当**选举未被触发**时，投票处理器会被条件变量阻塞 `rf.candidateCond.Wait()`，只能被选举处理器 `ticker()` 唤醒。

其次，如果所有的**投票任务还没结束**，投票处理器依然会被阻塞 `rf.wgForTicket.Wait()`。

所以，当投票处理器 `ticketer()` 会等待选举开始且所有的投票任务完成后，对投票结果判决。

- 如果当前Raft状态机**赢得选举**，将其角色更新为Leader，则广播心跳
- 如果当前Raft状态机**没有赢得选举**，将其角色更新为Follower

```
func (rf *Raft) ticketer() {
    for rf.killed() == false {
        rf.mu.Lock()
        for rf.state != CANDIDATE { rf.candidateCond.Wait() } // this
server isn't candidate. go wating.
        rf.mu.Unlock()

        rf.wgForTicket.Wait()

        if rf.isWin() {
            rf.mu.Lock()
            rf.state = LEADER

            rf.currentLeaderId = rf.me

            // rf.Start(nil) // no-op log entry
            rf.mu.Unlock()
        }
    }
}
```

```

        utils.Debugger(utils.DTrace, "S%d %s, elected as leader, %dth
term. %d servers agree this", rf.me, rf.state, rf.currentTerm,
atomic.LoadInt32(&rf.ticket))
        rf.leaderCond.Broadcast() // go live
    } else {
        rf.mu.Lock()
        rf.state = FOLLOWER
        rf.mu.Unlock()
    }
}
}

```

赢得选举的判定条件：只有当 `Candidate` 获的了超过一半的选票时，它才能赢得选举

```

func (rf *Raft) isWin() bool {
    votes := atomic.LoadInt32(&rf.ticket)
    return int(2 * votes) > len(rf.peers)
}

```

RequestVote RPC

角色为 `Candidate` 的Raft状态机投票请求发送给委员会中的所有 `Raft` 状态机，对应的 `Handler` 会被调用处理该请求。

投票请求中的请求参数：

- `Term int` : `Candidate` 的任期
- `CandidateId int` : `Candidate` 的id
- `LastLogIndex int` : `Candidate` 的最后一个日志记录的索引
- `LastLogTerm int` : `Candidate` 的最后一个日志记录的任期

```
type RequestVoteArgs struct {
    Term int // candidate's term
    CandidateId int // candidate requesting vote
    // election restriction
    LastLogIndex int // index of candidate's last log entry
    LastLogTerm int // term of candidate's last log entry
}
```

投票请求中的响应参数：

- `Term int`：请求接收方的任期
- `VoteGranted bool`：值为 `true` 表示接收方为该Candidate投票；否则接收方没有给该Candidate投票

```
type RequestVoteReply struct {
    Term int // current term, for candidate to update itself
    VoteGranted bool // true means candidate received vote
}
```

发送投票请求

`sendRequestVote` 方法的参数：`server int` 表示投票请求需要发送给名为 `server` 的Raft 状态机；`args` 是请求的参数；`reply` 请求的回复。

- 由于网络可能不可靠，出现请求丢失或响应丢失。如果没有收到来自接收方的响应，`sendRequestVote` 方法会最多尝试 `configs.MAXREQUESTCOUNT` 次重复发送请求。
- 由于 `rf.peers[server].Call` RPC是同步RPC调用，`sendRequestVote` 的调用者会处于阻塞状态，等待接受者的回复。

```

func (rf *Raft) sendRequestVote(server int, args *RequestVoteArgs,
reply *RequestVoteReply) bool {
    ok, count := false, 0
    for !ok { // ok = false, lose request or reply
        if count >= configs.MAXREQUESTCOUNT { break }
        utils.Debugger(utils.DVote, "S%d %s, sending vote request to S%d, %dth term", rf.me, rf.getState(), server, args.Term)
        ok = rf.peers[server].Call("Raft.RequestVote", args, reply) //
sync call
        count ++
    }
    if !ok {
        utils.Debugger(utils.DVote, "S%d %s, failed to vote request S%d, %dth term", rf.me, rf.getState(), server, args.Term)
    }
    return ok
}

```

处理投票请求

情况一： 如果RPC接收者的任期比 `Candidate` 任期高， **拒绝投票**

情况二： 已经投过票。如果RPC接收者的任期与 `Candidata` 任期相等（有可能是两个Raft同时发起选举，他们的任期可能会相同），并且该接受者已经给其他 `Candidate` 投过票了， **则拒绝投票**。

情况三： 重复请求。 如果RPC接收者的任期与 `Candidata` 任期相等，并且该接受者已经给当前的 `Candidate` 投过票了，则 **返回投票成功信息**。因为接收者返回的响应信息可能由于网络原因丢失。

情况四： 选举限制。 为了保证日志复制的安全性，Raft 要求只有 **包含所有已经提交日志记录的** `Candidate` 才能赢得选举。如果接受者持有的日志记录比 `Candidate` 的日志记录 **新**，则 **拒绝投票**。接收者如何判断自己的日志记录比 `Candidate` 的新？

- 如果两个Raft状态机的日志 `log` 中的最后一个日志记录的 **任期不同**，则 **任期越高** 的日志记录越新

- 如果两个Raft状态机的日志 `log` 中的最后一个日志记录的任期相同，则最后一个日志记录的索引越大越新

情况五：为 `Candidate` 投票。更新接受者以下信息：

- 重新设置选举定时器的超时时间；
- 更新接收者的任期 `currentTerm` 为 `Candidate` 的任期 `args.Term`；
- 更新 `rf.votedFor` 为 `args.CandidateId`；
- 将接收者角色转换成Follower

```
// example RequestVote RPC handler.
// if you have already voted in the current term, and an incoming
// RequestVote RPC has a higher term than you,
// you should first step down and adopt their term (thereby resetting
// votedFor),
// and then handle the RPC, which will result in you granting the
// vote!
func (rf *Raft) RequestVote(args *RequestVoteArgs, reply
*RequestVoteReply) {
    // time.Sleep(timeout(0,5))
    rf.mu.Lock()
    defer rf.mu.Unlock()
    defer rf.persist() // update rf.currentTerm and rf.votedFor

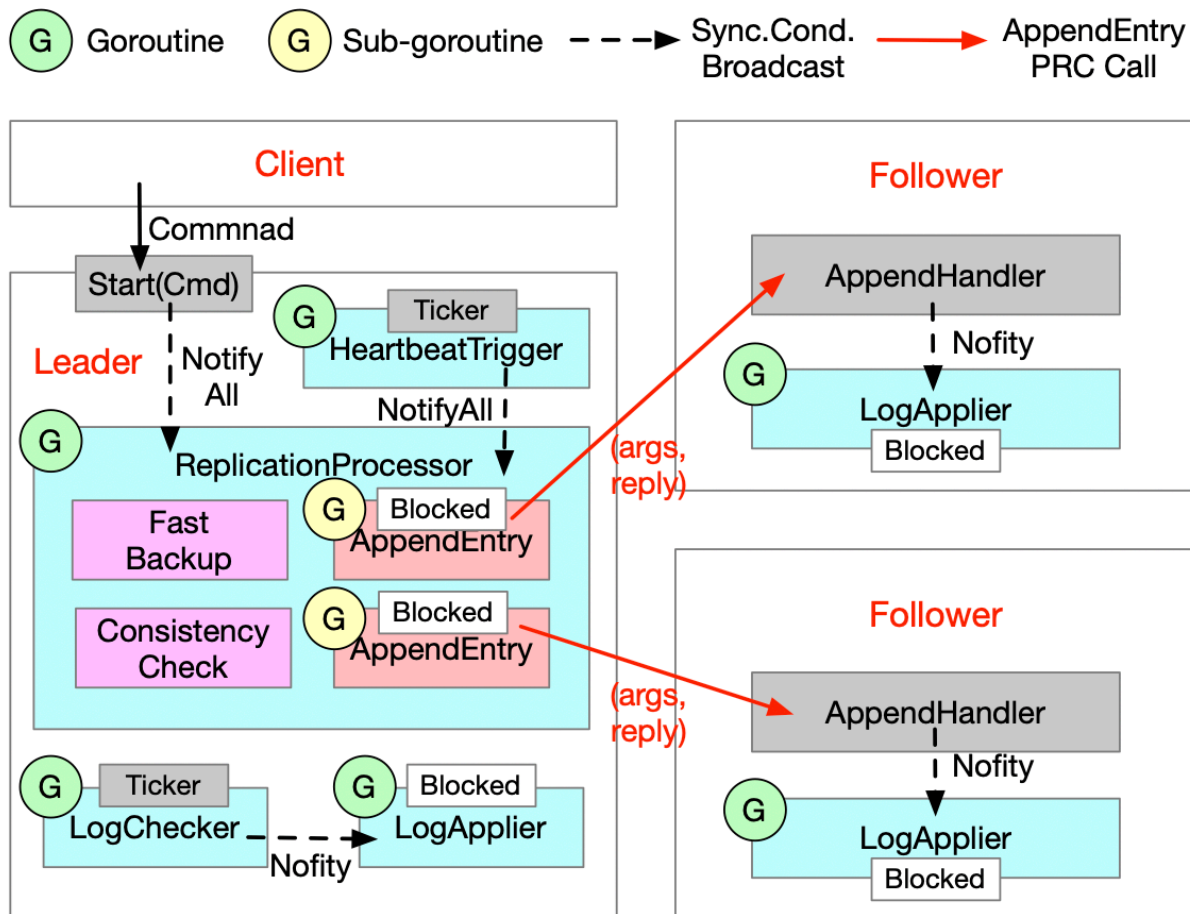
    if (rf.currentTerm > args.Term || (rf.votedFor != -1 &&
        rf.votedFor != args.CandidateId && rf.currentTerm == args.Term)) {
        reply.Term, reply.VoteGranted = rf.currentTerm, false // stale
        candidate or has voted in the term
    } else if rf.votedFor == args.CandidateId && rf.currentTerm ==
args.Term { // lose reply, client sends request again
        reply.Term, reply.VoteGranted = rf.currentTerm, true
    } else if ( (len(rf.log) == 0 && rf.lastIncludedTerm >
args.LastLogTerm) ||
        ( len(rf.log) == 0 && rf.lastIncludedTerm == args.LastLogTerm &&
rf.lastIncludedIndex > args.LastLogIndex ) ||
        ( len(rf.log) - 1 >= 0 && rf.log[len(rf.log) - 1].Term >
args.LastLogTerm) || // election restriction
```

```

        ( len(rf.log) - 1 >= 0 && rf.log[len(rf.log) - 1].Term ==
args.LastLogTerm && rf.sliceIndex2logIndex(len(rf.log) - 1) >
args.LastLogIndex)) {
    reply.Term, reply.VoteGranted = rf.currentTerm, false
    utils.Debugger(utils.DVote, "S%d %s, election restriction, refuse
to vote for S%d, %dth term.", rf.me, rf.state, args.CandidateId,
args.Term)
} else { //new term, vote
    utils.Debugger(utils.DVote, "S%d %s, voting for S%d, %dth term.",
rf.me, rf.state, args.CandidateId, args.Term)
    rf.timer.Reset(timeout(configs.ELECTION_TIME_START,
configs.ELECTION_TIME_END))
    rf.currentTerm, reply.Term = args.Term, args.Term
    reply.VoteGranted = true
    rf.votedFor = args.CandidateId
    rf.state = FOLLOWER // step down
}
}

```

日志复制



复制处理器 (ReplicationProcessor)

复制处理器是日志复制的核心组件。复制处理器 `trigger()` 是 **Raft构造器** 在构造Raft实例后，创建的一个单独的**后台协程**。复制处理器主要负责向Raft集群中的每一个Raft状态机发送日志复制请求 `AppendEntry RPC`。

由于使用同步RPC调用，为了实现高的并发，复制处理器会创建多个子协程。每个子协程负责发送日志复制请求 `AppendEntry RPC`，并阻塞等待响应信息。

- 所有的子协程一开始处于阻塞状态 `rf.triggerCond[server].Wait()`。当Raft状态机 (Leader) 接收到来自客户端的命令 **Command**，并将该命令封装成功日志记录追加到日志 `rf.log` 中时，会使用条件变量 `rf.triggerCond[i].Broadcast()` 唤醒 `trigger()` 的所有子协程发送日志记录负责请求，开始复制日志。
- 当Leader执行**心跳定时任务**时，所有的子协程会被唤醒，发送心跳包。

```
func (rf *Raft) trigger() {
    for i := 0; i < len(rf.peers); i++ {
```

```

if rf.me == i { continue }

go func(rf *Raft, server int) {
    for rf.killed() == false {
        rf.mu.Lock()
        rf.triggerCond[server].Wait()

        next := rf.nextIndex[server]
        // 初始化请求参数
        args := &RequestEntryArgs{
            Term:rf.currentTerm,
            LeaderId:rf.me,
            LeaderCommit:rf.commitIndex,
        }
        // 初始化响应参数
        reply := &RequestEntryReply{}
        // PrevLogIndex是Leader进行一致性检查的日志记录的索引
        args.PrevLogIndex = next - 1

        if rf.logIndex2sliceIndex(next - 1) == -1 { //
args.PrevLogIndex == rf.lastIncludeIndex
            args.PrevLogTerm = rf.lastIncludedTerm
        } else if rf.logIndex2sliceIndex(next - 1) < -1 { // follow-
server's log lagged leader, args.PrevLogIndex < rf.lastIncludeIndex
            utils.Debugger(utils.DInfo, "S%d %s, Send Snapshot to S%d,
rf.logIndex2sliceIndex(next - 1) %d, %dth term", rf.me, rf.state,
server, rf.logIndex2sliceIndex(next - 1), rf.currentTerm)
            rf.snapshotCond[server].Broadcast() // 当Follower落后Leader太
多时, Leader会发送快照给Follower进行同步
            rf.mu.Unlock()
            continue
        } else /*if rf.logIndex2sliceIndex(next - 1) < len(rf.log)*/ {
// args.PrevLogIndex > rf.lastIncludeIndex,
rf.logIndex2sliceIndex(next - 1) >= 0
            args.PrevLogTerm = rf.log[rf.logIndex2sliceIndex(next -
1)].Term
            // } else {

```

```

        // rf.logIndex2sliceIndex(next - 1) >= len(rf.log) //
nextIndex > len(rf.log)
    }

    if rf.logIndex2sliceIndex(next) < len(rf.log) { // Leader同步给
Follower的日志记录
        args.Entries = rf.log[rf.logIndex2sliceIndex(next):] // non-
heartbeat
    }
    rf.mu.Unlock()

    ch := make(chan bool)

    if !rf.isState(LEADER) { continue }

    go func(rf *Raft, server int, ch chan bool, args
*RequestEntryArgs, reply *RequestEntryReply) {
        ch <- rf.sendRequestEntry(server, args, reply)
    }(rf, server, ch, args, reply)

    select
    {
    case ok := <- ch:
        if(!ok) { continue } // ok==false, request/reply is
discarded
        case <-time.After(timeout(configs.REQUEST_TIMEOUT_START,
configs.REQUEST_TIMEOUT_END)): // dist server disconnect
            go func(ch chan bool) { // get the value from ch so that
this goroutine(sync-call-goroutine) can be destroyed
                <- ch
            }(ch)
            continue
        }

        if rf.isStaleServer(reply.Term) { continue } // check term,
stale server

        // 当一致性检查失败时，执行日志同步
        if !reply.Success { // reply failure

```

```

        // utils.Debugger(utils.DInfo, "S%d %s, S%d need backup,
rf.logIndex2sliceIndex(next - 1) %d, %dth term", rf.me, rf.getState(),
server, rf.logIndex2sliceIndex(next - 1), rf.currentTerm)
        rf.backup(server, reply.XTerm, reply.XIndex, reply.XLen)
        continue
    }

    rf.mu.Lock() // reply success
    // 日志复制成功时, 更新nextIndex为当前rf.log中最后一个日志记录的索引的
    下一位
    rf.nextIndex[server] = args.PrevLogIndex + 1 +
len(args.Entries) // rf.nextIndex[server] += len(args.Entries)
    // matchIndex[server]表示server的哪个索引的日志记录已经复制成功
    rf.matchIndex[server] = args.PrevLogIndex + len(args.Entries)
    // rf.matchIndex[server] = rf.nextIndex[server] - 1
    rf.mu.Unlock()
}
}(rf, i)
}
}

```

一致性检查 (ConsistencyCheck)

复制处理器在复制日志时依赖于一致性检查。

一致性检查的原理：数组 `rf.nextIndex[i]` 维护了 `Leader` 将要发送给 `Follower i` 的日志记录的下一个日志记录的索引。当一个 `Candidate` 当选为 `Leader` 后，其会初始化 `nextIndex` 的所有值为 `Leader` 日志数组 `log` 最后一个日志索引的下一位。

```

for i := 0; i < len(rf.peers); i ++ { // reinitialized nextIndex
    rf.nextIndex[i] = rf.sliceIndex2logIndex(len(rf.log)) //
sliceIndex2logIndex
    rf.matchIndex[i] = 0
}

```

为了使 follower 的日志跟自己 (leader) 一致, leader 必须找到 **两者达成一致的最大的日志条目索引**, 删除 follower 日志中从那个索引之后的所有日志条目, 并且将自己那个索引之后的所有日志条目发送给 follower。所有的这些操作都发生在 AppendEntries RPCs 的 **一致性检查的回复中**。

寻找 leader 与 follower 的最大公约数:

- leader 维护着一个针对每一个 follower 的 **nextIndex**, 这个 nextIndex 代表的就是 leader 要发送给 follower 的下一个日志条目的索引。
- 当选出一个新的 leader 时, 该 leader 将所有的 **nextIndex** 的值都初始化为自己最后一个日志条目的 **index 加 1** (图7 中的 11) 。
- 如果一个 follower 的日志跟 leader 的是 **不一致的**, 那么下一次的 AppendEntries RPC 的 **一致性检查就会失败**。
- **AppendEntries RPC 在被 follower 拒绝之后, leader 对 nextIndex 进行减 1, 然后重试 AppendEntries RPC。**
- 最终 **nextIndex** 会在某个位置满足 leader 和 follower 在该位置及之前的日志是一致的, 此时, **AppendEntries RPC 就会成功, 将 follower 跟 leader 冲突的日志条目全部删除然后追加 leader 中的日志条目 (需要的话)。**
- 一旦 AppendEntries RPC 成功, follower 的日志就和 leader 的一致了, 并且在该任期接下来的时间里都保持一致。

对上述一致性方法进行优化

如果需要的话, 下面的协议可以用来优化被拒绝的 AppendEntries RPCs 的个数(减少RPC请求次数)。

比如说, 当拒绝一个 AppendEntries RPC 的时候, follower 可以 **包含冲突条目的任期号和自己存储的那个任期的第一个 index**。借助这些信息, leader 可以跳过那个任期内所有的日志条目来减少 indexIndex。这样就变成了每个有冲突日志条目的任期只需要一个 AppendEntries RPC, 而不是每一个日志条目都需要一次 AppendEntires RPC。

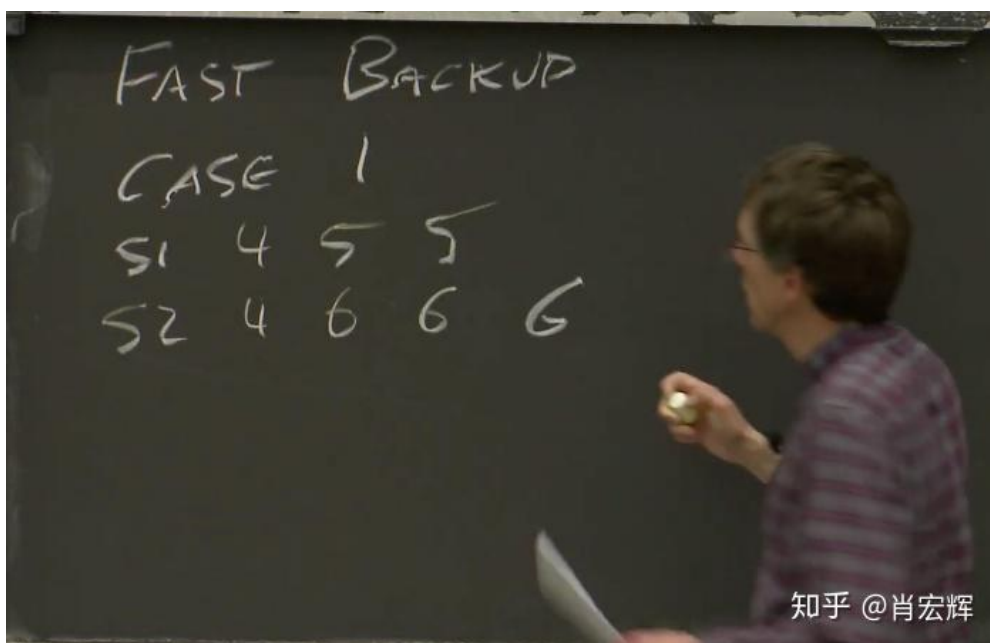
在实践中, 我们认为这种优化是没有必要的, 因为失败不经常发生并且也不可能有很多不一致的日志条目。

快速恢复 (FastBackup)

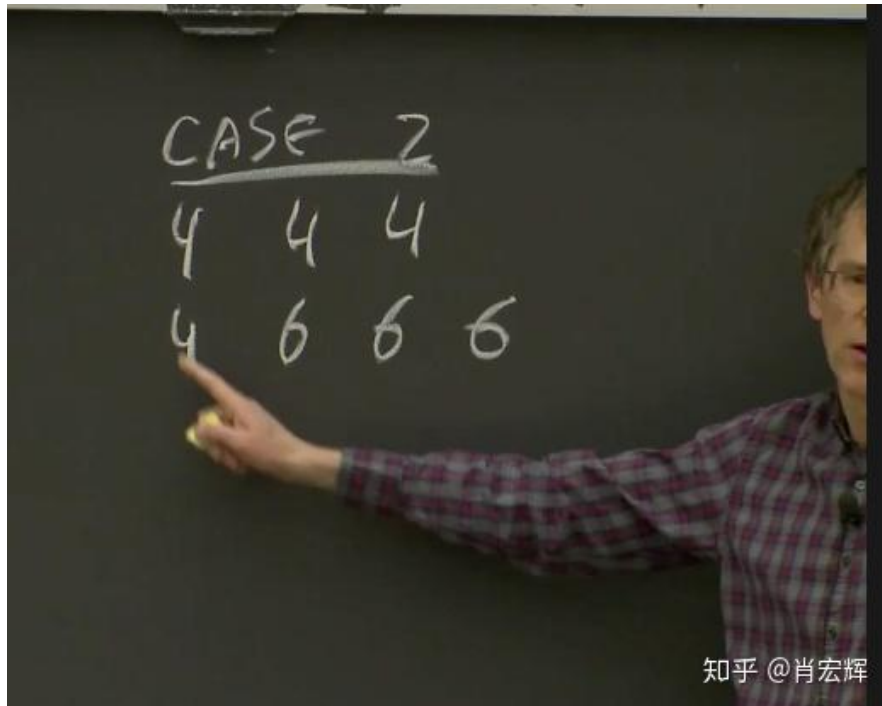
快速恢复的大致思想是，让 `Follower` 返回足够的信息给 `Leader`，这样 `Leader` 可以以任期 (**Term**) 为单位来回退，而不用每次只回退一条Log条目。所以现在，在恢复 `Follower` 的Log时，如果 `Leader` 和 `Follower` 的Log不匹配，`Leader` 只需要对每个不同的任期发送一条 `AppendEntries`，而不用对每个不同的Log条目发送一条 `AppendEntries`。这只是一种加速策略，当然，或许你也可以想出许多其他不同的日志恢复加速策略。

将可能出现的场景分成3类，为了简化，这里只画出一个Leader (S2) 和一个Follower (S1)，S2将要发送一条任期号为6的AppendEntries消息给Follower。

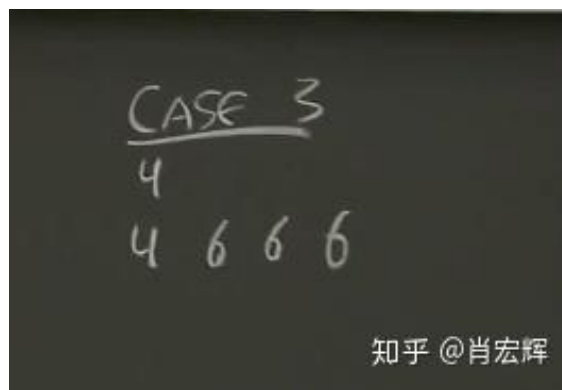
- 将可能出现的场景分成3类，为了简化，这里只画出一个Leader (S2) 和一个Follower (S1)，S2将要发送一条任期号为6的AppendEntries消息给Follower。



- 场景2：S1收到了任期4的旧Leader的多条Log，但是作为新Leader，S2只收到了一条任期4的Log。所以这里，我们需要覆盖S1中有关旧Leader的一些Log。



- 场景3：S1与S2的Log不冲突，但是S1缺失了部分S2中的Log。



可以让Follower在回复Leader的AppendEntries消息中，携带3个额外的信息，来加速日志的恢复。这里的回复是指，Follower因为Log信息不匹配，拒绝了Leader的AppendEntries之后的回复。这里的三个信息是指：

- **xTerm**：这个是Follower中与Leader冲突的Log对应的任期号。在之前有介绍Leader会在prevLogTerm中带上本地Log记录中，前一条Log的任期号。如果Follower在对应位置的任期号不匹配，它会拒绝Leader的AppendEntries消息，并将自己的任期号放在xTerm中。如果Follower在对应位置没有Log，那么这里会返回 -1。
- **xIndex**：这个是Follower中，对应任期号为xTerm的第一条Log条目的槽位号。
- **xLen**：如果Follower在对应位置没有Log，那么xTerm会返回-1，xLen表示空白的Log槽位数。

这些信息是如何在上面3个场景中，帮助Leader快速回退到适当的Log条目位置。

- 场景1。Follower (S1) 会返回XTerm=5, XIndex=2。Leader (S2) 发现自己没有任期5的日志，它会将自己本地记录的，S1的nextIndex设置到XIndex，也就是S1中，任期5的第一条Log对应的槽位号。所以，如果Leader完全没有XTerm的任何Log，那么它应该回退到XIndex对应的位置（这样，Leader发出的下一条AppendEntries就可以一次覆盖S1中所有XTerm对应的Log）。
- 场景2。Follower (S1) 会返回XTerm=4, XIndex=1。Leader (S2) 发现自己其实有任期4的日志，它会将自己本地记录的S1的nextIndex设置到本地在XTerm位置的Log条目后面，也就是槽位2。下一次Leader发出下一条AppendEntries时，就可以一次覆盖S1中槽位2和槽位3对应的Log。
- 场景3。Follower (S1) 会返回XTerm=-1, XLen=2。这表示S1中日志太短了，以至于在冲突的位置没有Log条目，Leader应该回退到Follower最后一条Log条目的下一条，也就是槽位2，并从这开始发送AppendEntries消息。槽位2可以从XLen中的数值计算得到。

当Leader的日志和Follower的日志不一致性时，Leader通过上述机制实现快速恢复。

```
// It is useful for bringing stale followers up to date quickl
func (rf *Raft) backup(server int, xterm int, xindex int, xlen int) {
    rf.mu.Lock()
    defer rf.mu.Unlock()
    if configs.FASTBACKUP { // fast backup
        if xterm == -1 { rf.nextIndex[server] -= xlen
        } else {
            l, _, ok := rf.bSearchRightBound(0, len(rf.log) - 1,
utils.Copy(rf.log).([]LogEntry), xterm)
            if ok {
                rf.nextIndex[server] = l + 1 // fix bugs:rf.nextIndex[server]
= int(math.Max(float64(l - 1), 1))
            } else {
                rf.nextIndex[server] = xindex
            }
        }
    } else { rf.nextIndex[server] -= 1 } // decrement nextIndex
}
```


AppendEntry RPC

日志追加请求中的请求参数：

`Term int`：Leader的任期

`LeaderId int`：Leader的Id

`PrevLogIndex int`：Leader的 `nextIndex[i] - 1`

`PrevLogTerm int`：索引为 `PrevLogIndex` 的日志记录的任期

`Entries []LogEntry`：Leader 将要发送给 Follower的日志记录

`LeaderCommit`：Leader已经提交的日志记录的索引

```
type RequestEntryArgs struct {
    Term int // leader's term
    LeaderId int // follower can redirect clients
    PrevLogIndex int // index of log entry immediately preceding new
ones
    PrevLogTerm int // term of prevLogIndex entry
    Entries []LogEntry // log entries to store (empty for heartbeat; may
send more than one for efficiency)
    LeaderCommit int // leader's commitIndex
}
```

日志追加请求中的响应参数：

- `Term` 是接收者的已经的最新任期
- `Success`： `true` 表示接收者为 `Candidate` 投票， `false` 表示拒绝投票
- `ServerId`：接收者的id

```

type RequestEntryReply struct {
    Term int // currentTerm, for leader to update itself
    Success bool // true if follower contained entry matching
prevLogIndex and prevLogTerm
    ServerId int // replier id
    Live bool //
    // Fast Backup
    XTerm int
    XIndex int
    XLen int
}

```

Leader 发送日志追加请求

```

func (rf *Raft) sendRequestEntry(server int, args *RequestEntryArgs,
reply *RequestEntryReply) bool {
    ok := rf.peers[server].Call("Raft.RequestEntry", args, reply)
    return ok
}

```

接收者处理日志追加请求

当请求参数中的 `Entries` 为空时，`AppendEntries` 是一个心跳包。当 `Entries` 不为空时，则 `AppendEntries` 是用于日志复制。

- **Leader合法性检查。** 如果 `rf.currentTerm > args.Term`，则Leader的任期号是旧的，接收者认为Leader是落单的，拒绝请求。如果 `rf.currentTerm <= args.Term`，则接收者认为发送日志复制请求的Leader是合法的，他会将自己的 `rf.currentLeaderId` 和 `rf.currentTerm` 更新为请求参数中的值（因为这个请求有可能是心跳包，接收者还会重新设置选举超时定时器的时间，将自己的角色更新为 `Follower`）。
- **日志一致性检查。** 接收者为了能够复制日志，接收者需要检查自己的日志与Leader的日志中哪些是不一致的，将不一致的日志记录删除，完成日志同步。

- 如果 `rf.sliceIndex2logIndex(len(rf.log) - 1) < args.PrevLogIndex`，接收者自己最后一个日志记录的索引小于Leader希望检查的日志记录的索引，则日志不一致。
- Leader希望检查的日志记录 `args.PrevLogIndex` 能够在接收者的日志 `log` 对应的索引位置中找到，但是日志记录的任期不一致，则日志不一致
- **过期的日志追加请求检查。** 网络延迟存在，导致网络中请求到达的时间不同。当Leader没有及时收到接收者的回复时，会重发请求。多个相同的请求可能会先后到达接受者，因此接收者可能需要检查重复的请求。
- **日志复制。** 接收者会将自己与Leader不一致的日志记录删除，然后复制Leader同步的日志记录。
- **日志提交。** 接收者收到Leader日志提交通知时，会提交 `[rf.lastApplied, args.LeaderCommit]` 之间的日志记录。

```
// Students' Guide to Raft(The importance of
details):https://thesquareplanet.com/blog/students-guide-to-raft/
func (rf *Raft) RequestEntry(args *RequestEntryArgs, reply
*RequestEntryReply) {
    rf.mu.Lock()
    defer rf.mu.Unlock()
    if rf.currentTerm > args.Term { // stale leader
        reply.Term = rf.currentTerm
        return
    }

    rf.currentLeaderId = args.LeaderId

    persistent := false
    if rf.currentTerm != args.Term {
        persistent = true
        defer rf.persist() // update term
    }

    rf.currentTerm, reply.Term, reply.Success = args.Term, args.Term,
false
    rf.timer.Reset(timeout(configs.ELECTION_TIME_START,
configs.ELECTION_TIME_END)) // reset timer
```

```

    if args.Entries == nil { utils.Debugger(utils.DDrop, "S%d %s,
Receive heartbeat from S%d", rf.me, rf.state, args.LeaderId) } //
heartbeats
    if rf.state == CANDIDATE { utils.Debugger(utils.DInfo, "S%d %s, Give
up election", rf.me, rf.state) } // candidate receives a heartbeat
from leader
    if rf.state == LEADER { utils.Debugger(utils.DInfo, "S%d %s, Stale
leader", rf.me, rf.state) } // leader receives a heartbeat from
another leader
    if args.Entries != nil { utils.Debugger(utils.DDrop, "S%d %s,
Receive entries from S%d", rf.me, rf.state, args.LeaderId) } //
entries

    rf.state = FOLLOWER

    if rf.sliceIndex2logIndex(len(rf.log) - 1) < args.PrevLogIndex { //
check inconsistency or consistency, sliceIndex2logIndex
        if configs.FASTBACKUP { reply.XTerm = -1; reply.XLen =
args.PrevLogIndex - rf.sliceIndex2logIndex(len(rf.log) - 1) } //
reply.XLen = args.PrevLogIndex - len(rf.log) + 1
        return
    }
    if (args.PrevLogIndex == rf.lastIncludedIndex && rf.lastIncludedTerm
!= args.PrevLogTerm) || // args.PrevLogIndex == rf.lastIncludedIndex
    ( rf.logIndex2sliceIndex(args.PrevLogIndex) >= 0 &&
rf.log[rf.logIndex2sliceIndex(args.PrevLogIndex)].Term !=
args.PrevLogTerm) { // args.PrevLogIndex > rf.lastIncludedIndex, check
inconsistency or consistency, logIndex2sliceIndex
        if configs.FASTBACKUP {
            reply.XTerm =
rf.log[rf.logIndex2sliceIndex(args.PrevLogIndex)].Term; //
logIndex2sliceIndex
            index, _, _ := rf.bSearchLeftBound(0,
rf.logIndex2sliceIndex(args.PrevLogIndex),
                utils.Copy(rf.log).([]*LogEntry),
rf.log[rf.logIndex2sliceIndex(args.PrevLogIndex)].Term)
            reply.XIndex = rf.sliceIndex2logIndex(index)
        }
    }

```

```

    return
}

if (args.PrevLogIndex < rf.lastIncludedIndex) { // args.PrevLogIndex
< rf.lastIncludedIndex, check if AppendEntries RPC is outdate
    utils.Debugger(utils.DWarn, "S%d %s, Outdate AppendEntries RPC,
The args.Entries[0] has been applied.", rf.me, rf.state)
    return
}

if args.Entries != nil { // && len(args.Entries) != 0
    // args.PrevLogIndex >= rf.lastIncludedIndex
    if len(args.Entries) <= (len(rf.log) -
(rf.logIndex2sliceIndex(args.PrevLogIndex) + 1)) { // check if
AppendEntries RPC is outdate
        var consistent = true
        for i := 0; i < len(args.Entries); i ++ {
            if (args.Entries[i].Term !=
rf.log[rf.logIndex2sliceIndex(args.PrevLogIndex + 1) + i].Term) {
                consistent = false
                break
            }
        }
        if consistent { // outdate AppendEntries RPC
            utils.Debugger(utils.DWarn, "S%d %s, Outdate AppendEntries
RPC, Follow has all the args.Entries.", rf.me, rf.state)
            reply.Success = true
            return
        }
    }
    // truncate
    rf.log =
utils.Copy(rf.log[:rf.logIndex2sliceIndex(args.PrevLogIndex) + 1]).
([]LogEntry) // delete
    rf.log = append(rf.log, utils.Copy(args.Entries).([]LogEntry)...)
    // append
    utils.Debugger(utils.DInfo, "S%d %s, Follower, Replicated entry,
index[%d, %d], term %d", rf.me, rf.state, args.PrevLogIndex + 1,
        rf.sliceIndex2logIndex(len(rf.log) - 1), rf.currentTerm)
    if !persistent { rf.persist() } // update log entries

```

```

}
// leaderCommit > len(rf.log) - 1 means all the follower's log
entries should have been committed.
if args.LeaderCommit > rf.commitIndex {
    rf.commitIndex = int(math.Min(float64(args.LeaderCommit), float64(
rf.sliceIndex2logIndex(len(rf.log) - 1) )))
    if (rf.commitIndex > rf.lastApplied) { rf.applyCond.Broadcast() }
// apply commands to state machine
}
reply.Success = true
}

```

心跳机制 (HeartbeatTrigger)

当 Candidate 当选 Leader 后，Leader 需要定时向 Follower 发送心跳包。心跳包的作用主要有两方面：

- 保持Leader的权威性。当 Follower 在选举超过时间过后，都没有收到来自 Leader的任何心跳信息，他认为Leader已经宕机，从而发起新一轮选举。因此，当Follower收到心跳信息后，会重置选举超时时间。
- 通过心跳信息，Leader能知道Raft集群中状态机的活性状态。
- 当日志不一致时，心跳包还起着日志复制的作用，以及通知Follower提交日志的作用

心跳触发器 `heartbeat()` 是 **Raft构造器** 在构造Raft实例后，创建的一个单独的**后台协程**。如果Raft状态机的角色不是Leader，心跳触发器会处于阻塞状态。当Raft状态机赢得选举，心跳触发器 `heartbeat()` 会被唤醒，定时发送心跳包。

采用 `time.NewTicker()` 创建一个**定时任务**，当定时器时间到后，会触发发送一次心跳包。

```

func (rf *Raft) heartbeat() {
    ticker := time.NewTicker(timeout(configs.HEARTBEAT_TIME_START,
configs.HEARTBEAT_TIME_END))
    for rf.killed() == false {

```

```

rf.mu.Lock()
for rf.state != LEADER { // this server isn't leader. go wating.
    rf.leaderCond.Wait()
    utils.Debugger(utils.DDrop, "S%d %s, Heartbeat, Go live", rf.me,
rf.state)
}
rf.mu.Unlock()
for i := 0; i < len(rf.peers); i ++ {
    rf.triggerCond[i].Broadcast()
}
<-ticker.C
}
}

```

半数复制策略 (LogChecker)

日志检查器 `checker()` 是 **Raft构造器** 在构造Raft实例后，创建的一个单独的**后台协程**。当Raft状态机赢得选举，日志检查器 `checker()` 会被唤醒，定时检查是否有日志记录已经复制到了超过一半的状态机上。

将日志检查视为一个**定时任务**，当定时时间到后，会从 `Leader` 的日志 `log` 的最后一个日志记录向前检查。

当检查到一个日志记录已经在**超过一半**的状态机上复制完成，Leader就会提交改日志记录及其之前的所有日志记录。

注意：Leader 无法直接提交非本任期的日志记录。

```

func (rf *Raft) checker() {
    for rf.killed() == false {
        rf.mu.Lock()
        // 该条日志记录不是本任期内的记录
        for rf.state != LEADER { rf.leaderCond.Wait() } // this server
isn't leader. go wating.
    }
}

```

```

    for n := len(rf.log) - 1; n >
rf.logIndex2sliceIndex(rf.commitIndex) && n >= 0 ; n -- { //
logIndex2sliceIndex
    if rf.log[n].Term != rf.currentTerm { continue } // A leader is
not allowed to update commitIndex to somewhere in a previous term

    count := 1 // leader
    for i := 0; i < len(rf.matchIndex); i ++ {
        if rf.me != i && rf.matchIndex[i] >= rf.sliceIndex2logIndex(n)
{ count ++ } // sliceIndex2logIndex
    }

    if 2 * count > len(rf.peers) { // 找到一条日志记录已经在大多数状态机
上完成复制
        if rf.sliceIndex2logIndex(n) > rf.lastApplied { //
sliceIndex2logIndex
            rf.commitIndex = rf.sliceIndex2logIndex(n) //
sliceIndex2logIndex
            utils.Debugger(utils.DInfo, "S%d Leader, Commit, index %d,
term %d", rf.me, rf.commitIndex, rf.currentTerm)
            rf.applyCond.Broadcast()
        }
        break
    }
}
rf.mu.Unlock()
time.Sleep(heartime(30))
}
}

```

日志提交器 (LogApplier)

日志提交器 `applier()` 是 **Raft构造器** 在构造Raft实例后，创建的一个单独的后台协程，默认处于阻塞状态。

- Leader 唤醒日志提交器：当日志检查器 `checker()` 检查到存在日志记录已经安全复制到状态机上，日志提交器 `applier()` 会被唤醒提交日志记录
- Follower 唤醒日志提交器：当 `AppendEntry Handler` 收到Leader的日志提交通知时，日志提交器 `applier()` 会被唤醒提交日志记录

`ApplyMsg` 类型，每当一条新的日志记录被提交后，Raft实例都应该向上层服务发送一个 `ApplyMsg` 的请求信息。`config` 的提交器监听到该请求后处理提交请求。

```
type ApplyMsg struct {
    CommandValid bool
    Command      interface{}
    CommandIndex int

    // For 2D:
    SnapshotValid bool
    Snapshot      []byte
    SnapshotTerm  int
    SnapshotIndex int
}
```

日志提交器会将每一条日志记录封装成 `ApplyMsg`，发送到 `rf.applyCh` 中。

```
func (rf *Raft) applier() {
    for rf.killed() == false {
        rf.mu.Lock()
        rf.applyCond.Wait()

        start := rf.lastApplied + 1
        end := rf.commitIndex
        log := utils.Copy(rf.log[rf.logIndex2sliceIndex(start):
rf.logIndex2sliceIndex(end + 1)]).([]LogEntry) // logIndex2sliceIndex
        utils.Debugger(utils.DInfo, "S%d %s, Apply entry, index [%d, %d],
term %d", rf.me, rf.state, start, end, rf.currentTerm)
        rf.lastApplied = rf.commitIndex
        rf.mu.Unlock()

        go func(rf *Raft, start int, log []LogEntry) {
            for i := 0; i < len(log); i ++ {
```

```

        rf.applyCh <- ApplyMsg{CommandValid:true,
Command:log[i].Command, CommandIndex:start + i} //
CommandIndex:logIndex
    }
    }(rf, start, log)
}
}

```

强一致性（或线性一致性）

Raft 日志复制 类似于两阶段提交 (two-phase commit) 来实现强一致性：

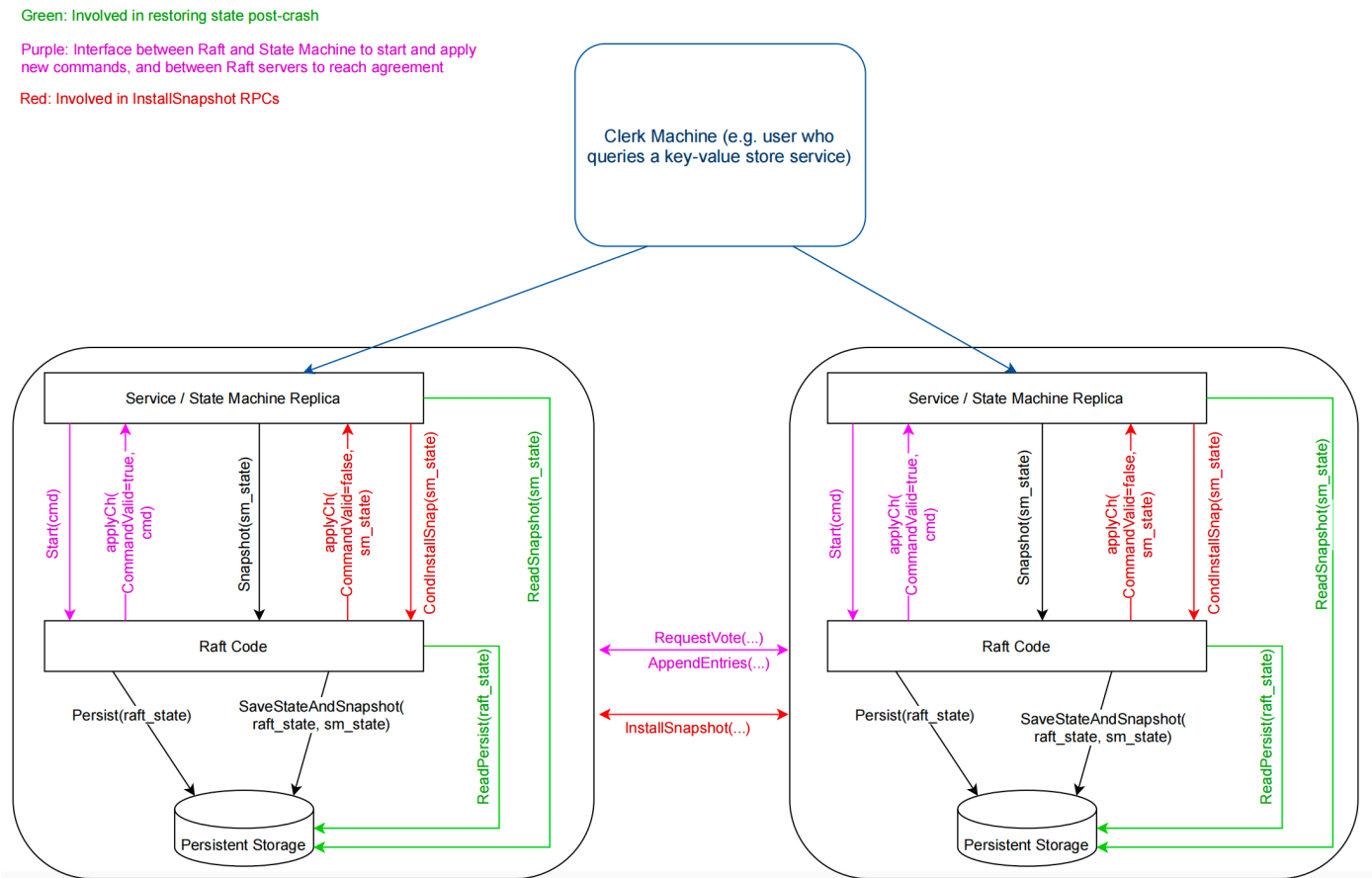
- 在第一个阶段，Leader 会向 Follower 发送日志记录追加请求。在追加日志记录之前，Raft 会进行协调Leader 与 Follower 通过 **多次请求完成一致性检查和日志同步**。此时，Follower不会实际执行日志记录中的命令。
- 在第二个阶段，如果 **超过一半** 的 Follower 都回复说可以执行该命令，这时 Leader才会通知Follower执行这些命名。

然而，Raft 采用 **半数票决系统** 保持强一致性，一条命令能够被认为执行完成当且仅当超过一半的服务器节点对Leader的远程过程调用进行了回应。

然而，大多数分布式系统采用 **两阶段提交Two-phase commit**，实现强一致性：

- 在第一个阶段，Primary向Secondary发请求，要求其执行某个操作，并等待Secondary回复说能否完成该操作，这时 **Secondary并不实际执行操作**。
- 在第二个阶段，如果所有Secondary都回复说可以执行该操作，这时Primary才会说，好的，所有Secondary执行刚刚你们回复可以执行的那个操作。

持久化和日志快照



持久化存储器（Persistent Storage）

采用 `Persister` 来模拟本地的持久化存储，`config` 在启动一个Raft状态机时，会为每一个Raft初始化一个持久化存储器 `Persister`。

数据结构：

- `raftstate []byte` 是Raft的状态进行持久化存储，例如 `log`
- `snapshot []byte` 是上层KV服务的状态快照

对外接口：

- `func MakePersister() *Persister`：初始化一个存储器
- `func (ps *Persister) Copy() *Persister`：复制一份存储器，浅拷贝

- `func (ps *Persister) ReadRaftState() []byte`：读取持久化存储的Raft的状态。`ReadRaftState()` 会被Raft构造器在实例化Raft时用于恢复Raft的状态数据。
- `func (ps *Persister) ReadSnapshot() []byte`：读取存储的KV服务的快照。
 - `ReadSnapshot` 会被Raft构造器在实例化Raft时初始化Snapshot本地缓存
 - `ReadSnapshot` 会被Raft的上层应用服务调用用于恢复上层服务的状态
- `func (ps *Persister) Save(raftstate []byte, snapshot []byte)`：持久化Raft状态和KV服务快照

```
//
// support for Raft and kvraft to save persistent
// Raft state (log &c) and k/v server snapshots.
//
type Persister struct {
    mu          sync.Mutex
    raftstate []byte
    snapshot []byte
}

func MakePersister() *Persister {
    return &Persister{}
}

func clone(orig []byte) []byte {
    x := make([]byte, len(orig))
    copy(x, orig)
    return x
}

func (ps *Persister) Copy() *Persister {
    ps.mu.Lock()
    defer ps.mu.Unlock()
    np := MakePersister()
    np.raftstate = ps.raftstate
    np.snapshot = ps.snapshot
}
```

```

    return np
}

func (ps *Persister) ReadRaftState() []byte {
    ps.mu.Lock()
    defer ps.mu.Unlock()
    return clone(ps.raftstate)
}

func (ps *Persister) RaftStateSize() int {
    ps.mu.Lock()
    defer ps.mu.Unlock()
    return len(ps.raftstate)
}

// Save both Raft state and K/V snapshot as a single atomic action,
// to help avoid them getting out of sync.
func (ps *Persister) Save(raftstate []byte, snapshot []byte) {
    ps.mu.Lock()
    defer ps.mu.Unlock()
    ps.raftstate = clone(raftstate)
    ps.snapshot = clone(snapshot)
}

func (ps *Persister) ReadSnapshot() []byte { // 被
    ps.mu.Lock()
    defer ps.mu.Unlock()
    return clone(ps.snapshot)
}

func (ps *Persister) SnapshotSize() int {
    ps.mu.Lock()
    defer ps.mu.Unlock()
    return len(ps.snapshot)
}

```

Raft State 持久化

持久化的存储可以确保当服务器重启时，服务器可以找到相应的数据，并将其加载到内存中。这样可以使得服务器在故障并重启后，继续重启之前的状态。

常见的故障

你或许会认为，如果一个**服务器故障了**，那简单直接的方法就是将它从集群中摘除。我们需要具备从集群中摘除服务器，替换一个全新的空的服务器，并让该新服务器在集群内工作的能力。实际上，这是至关重要的，因为如果一些服务器遭受了不可恢复的故障，例如**磁盘故障**，你绝对需要替换这台服务器。同时，如果磁盘故障了，你也不能指望能从该服务器的磁盘中获得任何有用的信息。所以我们的确需要能够用全新的空的服务器替代现有服务器的能力。你或许认为，这就足以应对任何出问题的场景了，但实际上不是的。

实际上，一个常见的**故障是断电**。断电的时候，整个集群都同时停止运行，这种场景下，我们不能通过从Dell买一些新的服务器来替换现有服务器进而解决问题。这种场景下，如果我们希望我们的**服务是容错的**，我们需要能够得到之前状态的拷贝，这样我们才能保持程序继续运行。因此，至少为了处理同时断电的场景，我们不得不让服务器能够将它们的状态存储在某处，这样**当供电恢复了之后，还能再次获取这个状态**。这里的**状态是指，为了让服务器在断电或者整个集群断电后，能够继续运行所必不可少的内容**。这是理解持久化存储的一种方式。

Raft 内置了 `persist()` 方法，能够将Raft的状态（`rf.log`、`rf.currentTerm` 和 `rf.votedFor`）进行持久化存储。使用用 `src/labgob` 序列化器将这些信息序列化成字节数组，然后持久化存储。有且仅有三个数据是需要持久化存储的。它们分别是 `Log`、`currentTerm`、`votedFor`。

- `Log`是所有的`Log`条目。当某个服务器刚刚重启，在它加入到Raft集群之前，它必须要检查并确保这些数据有效的存储在它的磁盘上。服务器必须要有某种方式来发现，自己的确有一些持久化存储的状态，而不是一些无意义的数字。**`Log`需要被持久化存储的原因是，这是唯一记录了应用程序状态的地方**。所以当服务器重启时，**唯一能用来重建应用程序状态的信息就是存储在`Log`中的一系列操作，所以`Log`必须要被持久化存储**。
- 那`currentTerm`呢？为什么`currentTerm`需要被持久化存储？是的，`currentTerm`和`votedFor`都是用来确保每个任期只有最多一个Leader。

- currentTerm的情况要更微妙一些，但是实际上还是为了 **实现一个任期内** 最多只有一个Leader，我们之前实际上介绍过这里的内容。如果（重启之后）我们不知道任期号是什么，很难确保一个任期内只有一个Leader，因为currentTerm需要用来保存已经被使用过的任期号。

这些数据需要在每次你修改它们的时候存储起来。所以可以确定的是，安全的做法是 **每次你添加一个Log条目，更新currentTerm或者更新votedFor**，你或许都需要持久化存储这些数据。

- 如果Leader收到了一个客户端请求，在发送AppendEntries RPC给Followers之前，必须要先持久化存储在本地。因为Leader必须要commit那个请求，并且不能忘记这个请求。实际上，在回复AppendEntries 消息之前，Followers也需要持久化存储这些Log条目到本地，因为它们最终也要commit这个请求，它们不能因为重启而忘记这个请求。

```
// require lock when it's invoked
func (rf *Raft) persist() {
    buffer := new(bytes.Buffer) // create empty buffer
    encoder := labgob.NewEncoder(buffer) // create encoder

    encoder.Encode(rf.log)
    encoder.Encode(rf.currentTerm)
    encoder.Encode(rf.votedFor)

    raftstate := buffer.Bytes() // []byte
    rf.persister.Save(raftstate, rf.snapshot) // save
    utils.Debugger(utils.DPersist, "S%d, Persists the raft state,
log:%v, term:%v, vote:%v", rf.me, rf.log, rf.currentTerm, rf.votedFor)
}
```

所以这（**优化磁盘写入性能**）是一个出现在所有系统中的常见的问题，也必然出现在Raft中，使得持久化存储就不再会是一个性能瓶颈

为什么服务器重启时，commitIndex、lastApplied、nextIndex、matchIndex，可以被丢弃？

- 之所以这些数据是非持久化存储的，是因为Leader可以通过检查自己的Log和发送给Followers的AppendEntries的结果，来发现哪些内容已经commit了。如果因为断电，所有节点都重启了。Leader并不知道哪些内容被commit了，哪些内容被执行了。但是当它发出AppendEntries，并从Followers搜集回信息。它会发现，Followers中有哪些Log与Leader的Log匹配，因此也就可以发现，在重启前，有哪些被commit了。

`readPersist()` 方法能够将字节数组 `data []byte` 反序列化成Raft 的状态变量，用于初始化Raft的状态。

```
// restore previously persisted state.
func (rf *Raft) readPersist(data []byte) {
    if data == nil || len(data) < 1 { return } // bootstrap without any
state?

    buffer := bytes.NewBuffer(data)
    decoder := labgob.NewDecoder(buffer)
    var log []LogEntry
    var term int
    var votedFor int
    if decoder.Decode(&log) != nil || decoder.Decode(&term) != nil ||
decoder.Decode(&votedFor) != nil {
        utils.Debugger(utils.DError, "S%d, Can't decode raft state!",
rf.me)
        return
    }

    rf.mu.Lock()
    rf.log = log
    rf.currentTerm = term
    rf.votedFor = votedFor
    rf.mu.Unlock()
    utils.Debugger(utils.DPersist, "S%d, Reads raft state form
persistent storage, log:%v, term:%v, vote:%v", rf.me, rf.log,
rf.currentTerm, rf.votedFor)
}
```


既然Log已经持久化存储了，那么应用程序状态就不必再持久化存储。因为Log从系统运行的初始就被持久化存储下来。所以，当Leader重启时，Leader会从第一条Log开始，执行每一条Log条目，并提交给应用程序。所以，重启之后，应用程序可以通过重复执行每一条Log来完全从头构建自己的状态。这是一种简单且优雅的方法，但是很明显会很慢。这将会引出我们的下一个话题：**Log compaction**和**Snapshot**。

Snapshot 持久化

在Raft中，Log压缩和快照解决的问题是：对于一个长期运行的系统，例如运行了几周，几个月甚至几年，如果我们按照Raft论文图2的规则，那么Log会持续增长。

- 最后可能会有数百万条Log，从而需要大量的内存来存储。
- 如果持久化存储在磁盘上，最终会消耗磁盘的大量空间。
- 如果一个服务器重启了，它需要通过重新从头开始执行这数百万条Log来重建自己的状态。当故障重启之后，遍历并执行整个Log的内容可能要花费几个小时来完成。

这在某种程度上来说是浪费，因为在重启之前，服务器已经有了一定的应用程序状态。

`Snapshot` 是 Raft 状态机对上层服务提供的接口，上层应用服务将快照 `snapshot []byte` 传输给Raft。

- Raft 更新快照的本地缓存信息
- 将快照持久化存储

快照的创建是否依赖应用程序。**快照生成函数是应用程序的一部分**，如果是一个key-value数据库，那么快照生成就是这个数据库的一部分。Raft会通过某种方式调用到应用程序，通知应用程序生成快照，因为只有应用程序自己才知道自己的状态（进而能生成快照）。而通过快照反向生成应用程序状态的函数，同样也是依赖应用程序的。

当Raft认为它的Log将会过于庞大，例如大于1MB，10MB或者任意的限制，Raft会要求应用程序在Log的特定位置，对其状态做一个快照。所以，**如果Raft要求应用程序做一个快照，Raft会从Log中选取一个与快照对应的点，然后要求应用程序在那个点的位置做一个快照**。这里极其重要，因为我们接下来将会丢弃所有那个点之前的Log

记录。如果我们有一个点的快照，那么我们可以安全的将那个点之前的Log丢弃。只要Raft持久化存储了快照，快照对应的Log槽位号，以及Log槽位号之后的所有Log，那么快照对应槽位号之前的这部分Log可以被丢弃，我们将不再需要这部分Log。

所以，Raft的持久化存储实际上是持久化应用程序快照，和快照之后的Log。

```
func (rf *Raft) Snapshot(index int, snapshot []byte) {
    // decode snapshot
    decodeBuffer := bytes.NewBuffer(snapshot)
    decoder := labgob.NewDecoder(decodeBuffer)
    var lastIncludedIndex int
    var serviceLayerState []interface{}
    if decoder.Decode(&lastIncludedIndex) != nil ||
        decoder.Decode(&serviceLayerState) != nil {
        utils.Debugger(utils.DError, "S%d, Can't decode snapshot!,
Snapshot", rf.me)
        return
    }
    if index != -1 && index != lastIncludedIndex {
        utils.Debugger(utils.DError, "S%d, Snapshot doesn't match
m.SnapshotIndex", rf.me)
        return
    }

    // re-encode snapshot
    rf.mu.Lock()
    if rf.logIndex2sliceIndex(index) < 0 ||
rf.logIndex2sliceIndex(index) >= len(rf.log){
        rf.mu.Unlock()
        return
    }
    utils.Debugger(utils.DSnap, "S%d %s, Start snapshot,
lastIncludedIndex %d, index %d, logIndex2sliceIndex(index) %d, %dth
term", rf.me, rf.state, rf.lastIncludedIndex, index,
rf.logIndex2sliceIndex(index), rf.currentTerm)

    rf.lastIncludedTerm = rf.log[rf.logIndex2sliceIndex(index)].Term
```

```

// 丢弃快照点之前的日志记录
rf.log = utils.Copy(rf.log[rf.logIndex2sliceIndex(index) + 1:]).
([]LogEntry) // discard old log entries
rf.serviceLayerState = serviceLayerState

rf.lastIncludedIndex = index

// meta data
encodeBuffer := new(bytes.Buffer)
encoder := labgob.NewEncoder(encodeBuffer)
encoder.Encode(rf.lastIncludedIndex)
encoder.Encode(rf.serviceLayerState)
encoder.Encode(rf.lastIncludedTerm)
rf.snapshot = clone(encodeBuffer.Bytes())

rf.persist()
rf.mu.Unlock()
}

```

`readSnapshot` 是Raft 内置的方法。用于从快照中 `snapshot []byte` 反序列化快照信息，初始化Raft的快照本地缓存。

```

func (rf *Raft) readSnapshot(snapshot []byte) {
    if snapshot == nil || len(snapshot) < 1{ return }

    rf.mu.Lock()
    defer rf.mu.Unlock()

    buffer := bytes.NewBuffer(snapshot)
    decoder := labgob.NewDecoder(buffer)
    var lastIncludedIndex int
    var serviceLayerState []interface{}
    var lastIncludedTerm int
    if decoder.Decode(&lastIncludedIndex) != nil ||
decoder.Decode(&serviceLayerState) != nil ||
    decoder.Decode(&lastIncludedTerm) != nil {
        utils.Debugger(utils.DError, "S%d, Can't decode snapshot!",
readSnapshot", rf.me)
        return
    }
}

```

```

}

rf.snapshot = clone(snapshot)
rf.lastIncludedIndex = lastIncludedIndex
rf.lastIncludedTerm = lastIncludedTerm
rf.serviceLayerState = serviceLayerState

rf.commitIndex = lastIncludedIndex
rf.lastApplied = lastIncludedIndex

utils.Debugger(utils.DSnap, "S%d %s, Read snapshot,
lastIncludedIndex %d, lastIncludedTerm %d, %dth term", rf.me,
rf.state, rf.lastIncludedIndex, rf.lastIncludedTerm, rf.currentTerm)
}

```

Snapshot 同步

同步器 (SnapshotSyncer)

不幸的是，这里丢弃了快照之前的Log，引入了大量的复杂性。如果有的Follower的Log较短，在Leader的快照之前就结束，那么除非有一种新的机制，否则那个Follower永远也不可能恢复完整的Log。

所以，Raft选择的方法是，Leader可以丢弃Follower需要的Log。所以，我们需要某种机制让AppendEntries能处理某些Follower Log的结尾到Leader Log开始之间丢失的这一段Log。解决方法是（一个新的消息类型）InstallSnapshot RPC。

- 当Follower刚刚恢复，如果它的Log短于Leader通过 AppendEntries RPC发给它的内容，那么它首先会强制Leader回退自己的Log。在某个点，**Leader将不能再回退**，因为它已经到了自己Log的起点。这时，**Leader会将自己的快照发给Follower**，之后立即通过AppendEntries将后面的Log发给Follower。

快照同步器 `syncer()` 是 **Raft构造器** 在构造Raft实例后，创建的一个单独的 **后台协程**，默认处于阻塞状态。在「日志复制」的一致性检查过程中，如果发生「Leader将不能再回退，因为它已经到了自己Log的起点」，快照同步器会被唤醒，并发送一个快照同步请求给Follower。

```
func (rf *Raft) syncer() {
```

```

for i := 0; i < len(rf.peers); i ++ {
    if rf.me == i { continue } // skip me

    go func(rf *Raft, server int) {
        for rf.killed() == false {
            rf.mu.Lock()
            rf.snapshotCond[server].Wait() // wait

            snapArgs := &RequestSnapshotArgs {
                Term: rf.currentTerm,
                LeaderId: rf.me,
                LastIncludedIndex: rf.lastIncludedIndex,
                LastIncludedTerm: rf.lastIncludedTerm,
                Data: rf.snapshot,
            }
            snapReply := &RequestSnapshotReply {}
            rf.mu.Unlock()

            snapCh := make(chan bool)

            if !rf.isState(LEADER) { continue }

            go func(rf *Raft, server int, snapCh chan bool, snapArgs
                *RequestSnapshotArgs, snapReply *RequestSnapshotReply) {
                snapCh <- rf.sendRequestSnapshot(server, snapArgs,
snapReply)
            }(rf, server, snapCh, snapArgs, snapReply)

            select
            {
            case ok := <- snapCh:
                if(!ok) { continue } // ok==false, request/reply is
discarded
            case <-time.After(timeout(configs.REQUEST_TIMEOUT_START,
configs.REQUEST_TIMEOUT_END)): // dist server disconnect
                go func(snapCh chan bool) { // get the value from ch so that
this goroutine(sync-call-goroutine) can be destroyed
                    <- snapCh
                }(snapCh)
            }
        }
    }
}

```

```

        continue
    }

    if rf.isStaleServer(snapReply.Term) { continue } // check
term, stale server

    rf.mu.Lock()
    rf.nextIndex[server] = snapArgs.LastIncludedIndex + 1
    rf.matchIndex[server] = snapArgs.LastIncludedIndex
    rf.mu.Unlock()
}
}(rf, i)
}
}

```

InstallShapshot RPC

快照同步请求的请求参数：

- `Term int`：Leader的任期
- `LeaderId int`：Leader的Id
- `LastIncludedIndex int`：快照点的日志记录的索引
- `LastIncludedTerm int`：快照点的日志记录的任期
- `Data []byte`：字节数组类型的快照数据

```

type RequestSnapshotArgs struct {
    Term int // leader's term
    LeaderId int // so follower can redirect clients
    LastIncludedIndex int // the snapshot replaces all entries up
through and including this index
    LastIncludedTerm int // term of lastIncludedIndex
    Offset int // byte offset where chunk is positioned in the snapshot
file
    Data []byte // raw bytes of the snapshot chunk, starting at offset
    Done bool // true if this is the last chunk
}

```

快照同步请求的响应参数：

```
type RequestSnapshotReply struct {  
    Term int // urrentTerm, for leader to update itself  
}
```

Leader 发送快照同步请求

```
func (rf *Raft) sendRequestSnapshot(server int, args  
*RequestSnapshotArgs, reply *RequestSnapshotReply) bool {  
    ok := rf.peers[server].Call("Raft.RequestSnapshot", args, reply)  
    return ok  
}
```

Follower 处理快照同步请求

- Follower需要忽略明显旧的快照。

```
func (rf *Raft) RequestSnapshot(args *RequestSnapshotArgs, reply  
*RequestSnapshotReply) {  
    rf.mu.Lock()  
    defer rf.mu.Unlock()  
    utils.Debugger(utils.DSnap, "S%d %s, Receive snapot,  
args.LastIncludedIndex %d, %dth term", rf.me, rf.state,  
args.LastIncludedIndex, rf.currentTerm)  
  
    if rf.currentTerm > args.Term { // stale leader  
        reply.Term = rf.currentTerm  
        return  
    }  
  
    rf.timer.Reset(timeout(configs.ELECTION_TIME_START,  
configs.ELECTION_TIME_END)) // reset timer
```

```

    if rf.state == CANDIDATE { utils.Debugger(utils.DInfo, "S%d %s, Give
up election, %dth term", rf.me, rf.state, rf.currentTerm) } //
candidate receives a heartbeat from leader

    rf.currentTerm, reply.Term = args.Term, args.Term
    rf.state = FOLLOWER

    if args.LastIncludedIndex > rf.sliceIndex2logIndex(len(rf.log) - 1)
{
    rf.log = make([]LogEntry, 0) // delete all log
} else if args.LastIncludedIndex > rf.lastIncludedIndex {
    rf.log =
utils.Copy(rf.log[rf.logIndex2sliceIndex(args.LastIncludedIndex) +
1:]).([]LogEntry) // delete log entries that are covered by snapshot
}

    if args.LastIncludedIndex > rf.sliceIndex2logIndex(len(rf.log) - 1)
|| args.LastIncludedIndex > rf.lastIncludedIndex { // lagged
    rf.lastIncludedIndex = args.LastIncludedIndex
    rf.lastIncludedTerm = args.LastIncludedTerm
    rf.snapshot = clone(args.Data)

    rf.commitIndex = args.LastIncludedIndex
    rf.lastApplied = args.LastIncludedIndex

    go func(rf *Raft, data []byte , index int, term int) { // re-bulid
state of machine state by snapshot
        utils.Debugger(utils.DSnap, "S%d %s, Apply snapshot, %dth term",
rf.me, rf.state, rf.currentTerm)
        rf.applyCh <- ApplyMsg{SnapshotValid:true, Snapshot: data,
SnapshotIndex:index, SnapshotTerm:term}
    }(rf, args.Data, args.LastIncludedIndex, args.LastIncludedTerm)

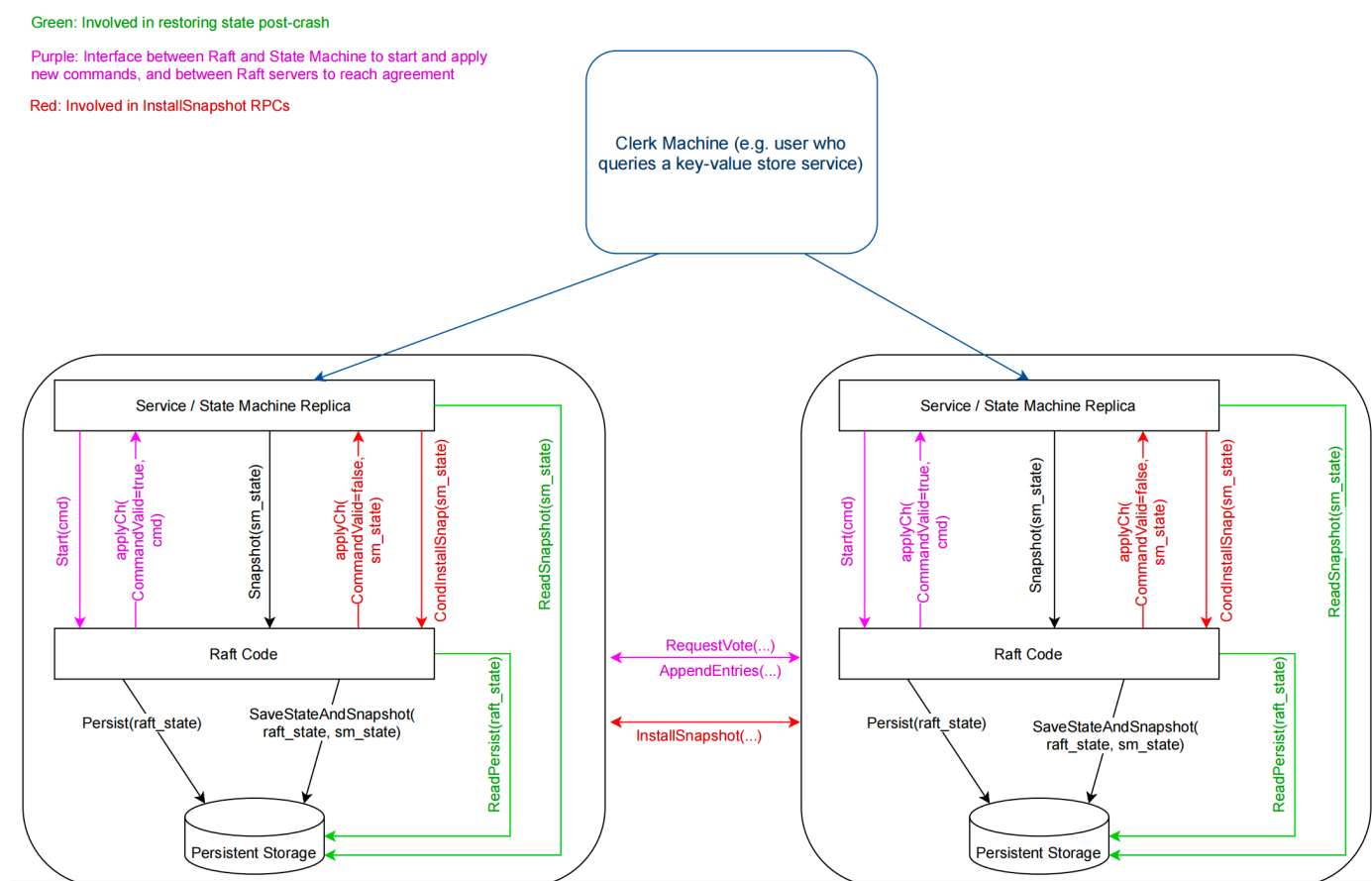
    rf.persist() // persist
}
}

```


Raft KV 服务

什么是 Raft KV 服务?

Raft KV 服务主要三部分组件构成：Clerk、Service 和 Raft。其中，Raft在之前的内容中已经实现，KV Service是一个可复制的状态机，由许多通过Raft实现复制的KV 服务器组成。当遇到一些故障或网络分区是，只要超过一半的KV服务器处于活性状态并且能够互相通信，那么KV Service 就能够持续处理来自Clerk的请求。



KV Server 维护了一个简单的键值对内存数据库。键和值的类型都是字符串类型。其中，Clerk 能发生三种不同的RPC给 KV Service：

- `Put(key, value)`：修改或新增内存数据库中给定 Key 对应的 Value
- `Append(key, value)`：向内存数据库中Key对的值字符串后面追加字符串 **Value**。如果键不存在，则该操作退化为Put(key, value)操作。

- `Get(key)`：从内存数据库中查询Key对应的值。如果查询的Key不存在，返回一个空字符串。

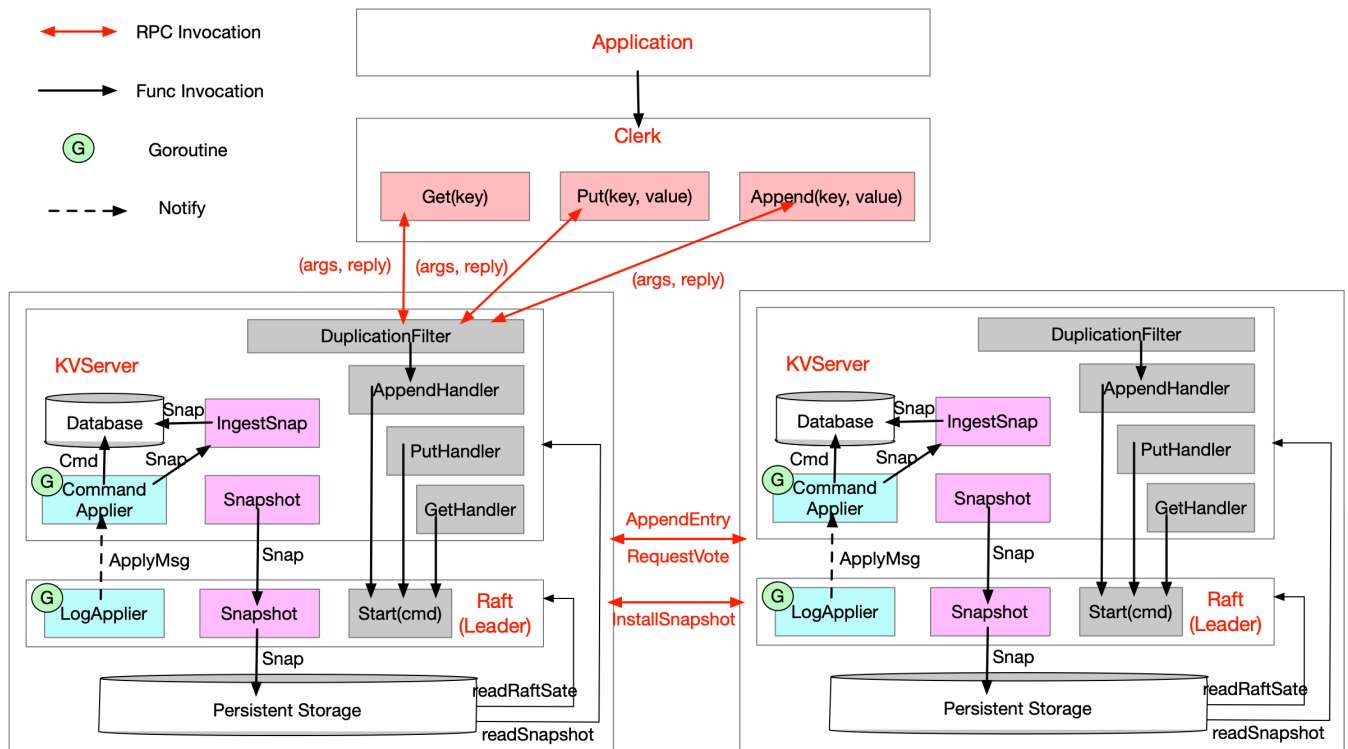
外应用需要调用 Clerk 提供的 `Put/Append/Get` 方法才能访问KV Service，而Clerk 管理了所有与Server通信的RPCs。

KV 服务应该保持强一致性。 KV 服务必须排列应用程序对Clerk `Get/Put/Append`方法的调用是线性化的。如果一次只调用一个方法，那么`Get/Put/Append`方法的行为就好像系统中只有一个状态副本，而且每次调用都应该能观察前面的调用序列对状态的修改。

对于并发调用，返回值和最终状态必须与按某种顺序一次执行一个操作的情况相同。如果客户端X调用了`Clerk.Put()`，客户端Y调用了`Clerk.Append()`，那么客户端X的调用就会返回。在调用开始之前，必须观察所有已经完成的调用的执行效果。

线性化对于应用程序来说很方便，因为它是你在单个服务器一次处理一个请求的行为。例如，如果一个客户端成功地从服务获得更新请求的响应，那么随后从其他客户端启动的读取保证可以看到该更新的效果。对于单个服务器来说，提供线性化相对容易。如果服务是复制的，那就更难了，因为所有服务器必须为并发请求选择相同的执行顺序，必须避免使用不是最新的状态回复客户端，并且必须在失败后以保留所有已确认的客户端更新的方式恢复它们的状态。

Raft KV 运行流程是什么？



每个键值对服务器(“kvserver”)都有一个关联的Raft节点。Clerk向kvserver发送Put()、Append()和Get() RPC请求，其关联的Raft是leader。kvserver代码向Raft提交Put/Append/Get操作，以便Raft日志保存一系列Put/Append/Get操作。所有kvserver按顺序执行Raft日志中的操作，将操作应用到它们的键值对数据库。这样做的目的是让服务器维护键值对数据库的相同副本。

Clerk有时不知道哪个kvserver是Raft Leader。如果Clerk发送一个RPC到错误的kvserver，或者它不能到达kvserver，Clerk应该通过发送到不同的kvserver重新尝试。如果键值对服务将操作提交到它的Raft日志(因此将操作应用到键值对状态机)，那么leader通过响应其RPC将结果报告给Clerk。如果操作未能提交(例如，leader被替换了)，服务器会报告一个错误，然后Clerk会用另一个服务器重新尝试。

所有的kvserver之间不允许直接通信，他们互相通过Raft进行交互。

Clerk

应用程序Application能够通过 Clerk 去访问 KVServer 。因此， Clerk 就相当于 KVServer 的客户端。

- `servers []*labrpc.ClientEnd` : Clerk 采用RPC与 KVServer 进行通信，所以 Clerk 需要持有所有能够与 kvservers 通信的RPC客户端 `ClientEnd` 。
- `cid` : 客户端的唯一id。在分布式环境下， `cid` 是一个分布式ID。可以采用雪花算法生成一个分布式ID。
- `rid` : 客户端发送请求的请求ID，rid从0开始递增，可以不是分布式ID。

```
// Each client talks to the service through a Clerk with
Put/Append/Get methods. A Clerk manages RPC interactions with the
servers.
// Each of your key/value servers ("kvservers") will have an
associated Raft peer.
// Clerks send Put(), Append(), and Get() RPCs to the kvserver whose
associated Raft is the leader.
type Clerk struct {
    mu sync.Mutex
    servers []*labrpc.ClientEnd
    randomizer *random.Rand

    cid int64 // a unique client ID, perhaps a 64-bit random number
    rid int64 // a sequence number of request.
}
```

构造方法

初始化 `cid` 和 `rid`

```
func MakeClerk(servers []*labrpc.ClientEnd) *Clerk {
    ck := new(Clerk)

    ck.servers = servers
    ck.randomizer = random.New(random.NewSource(time.Now().UnixNano()))
    // 使用当前时间戳 生成真正的随机数字

    ck.cid = nrand()
    ck.initRid()
    // ck.lastUIDs = make(map[int64]bool)

    return ck
}
```

RID

```
func (ck *Clerk) initRid() {
    ck.mu.Lock()
    defer ck.mu.Unlock()
    ck.rid = 0
}

func (ck *Clerk) getRid() int64 {
    ck.mu.Lock()
    defer ck.mu.Unlock()
    ck.rid += 1
    return ck.rid
}
```

Get(key) 从内存数据库中取出Key对应的值：

- 如果Key对应的值不存在，则返回空字符串。
- 如果由于网络不可靠或者KVServer发生故障等问题，没有得到响应失败，重新发起Get PRC请求。

```

// What should a client do if a Put or Get RPC times out?
//   i.e. Call() returns false
//   if server is dead, or request was dropped: re-send
//   if server executed, but reply was lost: re-send is dangerous
// A Clerk sometimes doesn't know which kvserver is the Raft leader.
// If the Clerk sends an RPC to the wrong kvserver, or if it cannot
reach the kvserver,
// the Clerk should re-try by sending to a different kvserver.

// fetch the current value for a key.
// returns "" if the key does not exist.
// keeps trying forever in the face of all other errors.
//
// you can send an RPC with code like this:
// ok := ck.servers[i].Call("KVServer.Get", &args, &reply)
//
// the types of args and reply (including whether they are pointers)
// must match the declared types of the RPC handler function's
// arguments. and reply must be passed as a pointer.
func (ck *Clerk) Get(key string) string {
    leaderId := ck.randomizer.Intn(len(ck.servers))

    args := &GetArgs{
        Op: GET,
        Key: key,
        RID: ck.getRid(), // same ID in re-sends of same RPC
        CID: ck.cid,
    }

    for {
        reply := &GetReply{}
        utils.Debugger(utils.DInfo, "S, Leader may be %d.", leaderId)

        ch := make(chan bool)
        go func(ck *Clerk, args *GetArgs, reply *GetReply, leaderId int,
ch chan bool) {
            ok := ck.servers[leaderId].Call("KVServer.Get", args, reply) //
sync call
            ch <- ok

```

```

}(ck, args, reply, leaderId, ch)

ok := false
select // timer
{
    case ok = <- ch: // ok whose value is false means that request or
reply is discarded
        case <-time.After(timeout(configs.KVRAFT_REQUEST_TIMEOUT_START,
configs.KVRAFT_REQUEST_TIMEOUT_END)): // dist server disconnect or
long delay
            ok = false
            go func(ch chan bool) { // get the value from ch so that this
goroutine(sync-call-goroutine) can be destroyed
                <- ch
            }(ch)
        }

    if ok && reply.Err == OK { // success
        utils.Debugger(utils.DInfo, "GET Reponse, sucess, key '%s',
value '%s'.", key, reply.Value)
        return reply.Value
    }

    if ok && reply.Err == ErrNoKey { // no key
        utils.Debugger(utils.DInfo, "GET Reponse, no key.")
        return ""
    }

    if !ok { // failed rpc, can't reach kvserver, drop reply
        leaderId = ck.randomizer.Intn(len(ck.servers)) // select a
server randomly
    } else if reply.Err == ErrWrongLeader { // wrong leader
        leaderId = reply.LeaderId
        utils.Debugger(utils.DInfo, "GET Reponse, Wrong Leader, Leader
may be %d.", leaderId)
    }
}
return ""
}

```

`PutAppend(key, value, op)` 修改内存数据库的值。 `Put(key,value)` 和 `Append(key,value)` 可以复用相同的RPC请求。 类似地, 如果由于网络不可靠或者KVServer发生故障等问题, 没有得到响应失败, 重新发起 `PutAppend(key, value, op)` PRC请求。

```
// shared by Put and Append.
//
// you can send an RPC with code like this:
// ok := ck.servers[i].Call("KVServer.PutAppend", &args, &reply)
//
// the types of args and reply (including whether they are pointers)
// must match the declared types of the RPC handler function's
// arguments. and reply must be passed as a pointer.
func (ck *Clerk) PutAppend(key string, value string, op string) {
    leaderId := ck.randomizer.Intn(len(ck.servers))

    args := &PutAppendArgs{
        Key:key,
        Value:value,
        Op:op,
        RID:ck.getRid(),
        CID:ck.cid,
    }

    for {
        reply := &PutAppendReply{}
        utils.Debugger(utils.DInfo, "S, Leader may be %d", leaderId)

        ch := make(chan bool)
        go func(ck *Clerk, args *PutAppendArgs, reply *PutAppendReply,
            leaderId int, ch chan bool) {
            ok := ck.servers[leaderId].Call("KVServer.PutAppend", args,
            reply) // sync call
            ch <- ok
        }(ck, args, reply, leaderId, ch)
```



```

    ok := false
    select // timer
    {
        case ok = <- ch: // ok whose value is false means that request or
reply is discarded
            case <-time.After(timeout(configs.KVRAFT_REQUEST_TIMEOUT_START,
configs.KVRAFT_REQUEST_TIMEOUT_END)): // dist server disconnect or
long delay
                ok = false
                go func(ch chan bool) {
                    <-ch
                }(ch)
            }

        if ok && reply.Err == OK { // success
            return
        }

        if !ok { // failed rpc, can't reach kvserver, drop reply
            leaderId = ck.randomizer.Intn(len(ck.servers)) // select a
server randomly
        } else if reply.Err == ErrWrongLeader { // wrong leader
            leaderId = reply.LeaderId
            utils.Debugger(utils.DInfo, "%s Reponse, wrong leader, leader
may be S%d.", op, leaderId)
        }
    }
}

```

KVServer

KVServer 数据结构

```
type KVServer struct {
    mu      sync.Mutex
    me      int
    rf      *raft.Raft
    applyCh chan raft.ApplyMsg
    dead    int32 // set by Kill()

    maxraftstate int // snapshot if log grows this big

    persister *raft.Persister // Object to hold this peer's persisted
state

    database map[string]string // kv table
    duplicate map[int64]Result // (cid, operation result), k/v service
maintains a "duplicate table" indexed by ID

    cond *sync.Cond
}
```

构造方法

```
// servers[] contains the ports of the set of
// servers that will cooperate via Raft to
// form the fault-tolerant key/value service.
// me is the index of the current server in servers[].
// the k/v server should store snapshots through the underlying Raft
// implementation, which should call persister.SaveStateAndSnapshot()
to
// atomically save the Raft state along with the snapshot.
// the k/v server should snapshot when Raft's saved state exceeds
maxraftstate bytes,
// in order to allow Raft to garbage-collect its log. if maxraftstate
is -1,
// you don't need to snapshot.
// StartKVServer() must return quickly, so it should start goroutines
```

```
// for any long-running work.
func StartKVServer(servers []*labrpc.ClientEnd, me int, persister
*raft.Persister, maxraftstate int) *KVServer {
    // call labgob.Register on structures you want
    // Go's RPC library to marshall/unmarshall.
    labgob.Register(Op{})
    labgob.Register(make(map[string]string)) // register map, reference
https://www.jb51.net/article/211523.htm
    labgob.Register(make(map[int64]Result))

    kv := new(KVServer)
    kv.me = me
    kv.maxraftstate = maxraftstate
    kv.applyCh = make(chan raft.ApplyMsg)
    kv.rf = raft.Make(servers, me, persister, kv.applyCh)
    kv.persister = persister

    // You may need initialization code here.
    kv.database = make(map[string]string)
    kv.duplicate = make(map[int64]Result)
    kv.cond = sync.NewCond(&kv.mu)

    kv.loadSnapshot(persister.ReadSnapshot()) // ReadSnapshot

    go kv.applier()

    return kv
}
```

操作内存数据库

```
func (kv *KVServer) get(key string) (string, bool) { // get value
based on key from kv.database securely
    utils.Debugger(utils.DInfo, "S%d, apply, GET, key '%s'.", kv.me,
key)
    kv.mu.Lock()
```

```

    value, ok := kv.database[key]
    kv.mu.Unlock()
    return value, ok
}

func (kv *KVServer) put(key string, value string) { // put (key,
value) to kv.database securely
    utils.Debugger(utils.DInfo, "S%d, apply, PUT, key '%s', value
'%s'.", kv.me, key, value)
    kv.mu.Lock()
    kv.database[key] = value
    kv.mu.Unlock()
}

func (kv *KVServer) append(key string, value string) { // append (key,
value) to kv.database securely
    utils.Debugger(utils.DInfo, "S%d, apply, APPEND, key '%s', value
'%s'.", kv.me, key, value)
    kv.mu.Lock()
    v, ok := kv.database[key]
    if ok {
        kv.database[key] = v + value
    } else {
        kv.database[key] = value
    }
    kv.mu.Unlock()
}

```

RPC Handler

```

func (kv *KVServer) Get(args *GetArgs, reply *GetReply) {
    _, isLeader := kv.rf.GetState()

    if !isLeader { // return directly when current raft's server is not
leader

```

```

    reply.Err = ErrWrongLeader
    reply.LeaderId = kv.rf.GetCurrentLeaderId()
    return
}

kv.mu.Lock()
res, ok := kv.duplicate[args.CID] // check if the command whose uid
is args.UID has executed.
kv.mu.Unlock()

if !ok || args.RID > res.RID {
    op := Op{
        Operation:args.Op,
        Key:args.Key,
        CID:args.CID,
        RID:args.RID,
    }

    _, _, _ = kv.rf.Start(op)
    utils.Debugger(utils.DLeader, "Leader %d, Get, key '%s'.", kv.me,
args.Key)

    kv.mu.Lock() // wait
    res, ok = kv.duplicate[args.CID]
    for !ok || args.RID != res.RID {
        kv.cond.Wait()
        res, ok = kv.duplicate[args.CID]
    }
    kv.mu.Unlock()
}

reply.LeaderId = kv.rf.GetCurrentLeaderId()

if res.Err { // No Key
    reply.Err = ErrNoKey
    return
}

reply.Err = OK // success
reply.Value = res.Value

```

```

}

func (kv *KVServer) PutAppend(args *PutAppendArgs, reply
*PutAppendReply) {
    _, isLeader := kv.rf.GetState()

    if !isLeader { // return directly when current raft's server is not
leader
        reply.Err = ErrWrongLeader
        reply.LeaderId = kv.rf.GetCurrentLeaderId()
        return
    }

    kv.mu.Lock()
    res, ok := kv.duplicate[args.CID] // check if the command whose uid
is args.UID has executed.
    kv.mu.Unlock()

    if !ok || args.RID > res.RID {
        op := Op{
            Operation:args.Op,
            Key:args.Key,
            Value:args.Value,
            CID:args.CID,
            RID:args.RID,
        }
        _, _, _ = kv.rf.Start(op)
        utils.Debugger(utils.DLeader, "S%d Leader, Put/Append, key '%s',
value '%s'.", kv.me, args.Key, args.Value)

        kv.mu.Lock() // wait
        res, ok = kv.duplicate[args.CID]
        for !ok || args.RID != res.RID {
            kv.cond.Wait()
            res, ok = kv.duplicate[args.CID]
        }
        kv.mu.Unlock()
    }
}

```

```

reply.Err = OK
reply.LeaderId = kv.rf.GetCurrentLeaderId()
}

```

重复请求检测策略

CommandApplier

```

func (kv *KVServer) applier() {
    for !kv.killed() {
        m := <- kv.applyCh

        if m.CommandValid {
            if m.Command == nil { continue }
            op := m.Command.(Op)

            kv.mu.Lock()
            res, ok := kv.duplicate[op.CID] // check if the command whose
uid is args.UID has executed.
            kv.mu.Unlock()
            if ok && res.RID == op.RID {
                kv.cond.Broadcast()
                continue
            }

            if op.Operation == GET {
                value, ok := kv.get(op.Key)
                kv.setDuplicate(op.CID, Result{RID:op.RID , Err:!ok,
Value:value})
                kv.broadcast() // wake up
            } else if op.Operation == PUT {

```

```

        kv.put(op.Key, op.Value)
        kv.setDuplicate(op.CID, Result{RID:op.RID , Err:false,
Value:""})
        kv.broadcast() // wake up
    } else if op.Operation == APPEND {
        kv.append(op.Key, op.Value)
        kv.setDuplicate(op.CID, Result{RID:op.RID, Err:false,
Value:""})
        kv.broadcast() // wake up
    }

    if kv.maxraftstate != -1 && kv.persister.RaftStateSize() >
kv.maxraftstate { // snapshot. If maxraftstate is -1, we do not have
to snapshot
        // fmt.Printf("Beofore Snapshot,
kv.persister.RaftStateSize()=%d\n", kv.persister.RaftStateSize())
        kv.snapshot(m.CommandIndex)
    }

} else if m.SnapshotValid {
    kv.ingestSnap(m.Snapshot, m.SnapshotIndex)
    utils.Debugger(utils.DInfo, "ingestSnap")
}
}
}

```

Snapshot

```

func (kv *KVServer) snapshot(CommandIndex int) { // monitor if
persister.RaftStateSize() is greater than maxraftstate
    var serviceLayerState []interface{} // serviceLayerState
    kv.mu.Lock()
    serviceLayerState = append(serviceLayerState, kv.database)
    serviceLayerState = append(serviceLayerState, kv.duplicate)
    kv.mu.Unlock()
}

```



```

buffer := new(bytes.Buffer)
encoder := labgob.NewEncoder(buffer)
encoder.Encode(CommandIndex)
encoder.Encode(serviceLayerState)

kv.rf.Snapshot(CommandIndex, buffer.Bytes())
// fmt.Printf("After Snapshot, kv.persister.RaftStateSize()=%d\n",
kv.persister.RaftStateSize())
}

func (kv *KVServer) ingestSnap(snapshot []byte, index int) {
    if snapshot == nil {
        utils.Debugger(utils.DError, "nil snapshot in
src/kvraft/server.go")
        return
    }
    buffer := bytes.NewBuffer(snapshot)
    decoder := labgob.NewDecoder(buffer)
    var lastIncludedIndex int
    var serviceLayerState []interface{}

    if decoder.Decode(&lastIncludedIndex) != nil ||
        decoder.Decode(&serviceLayerState) != nil {
        utils.Debugger(utils.DError, "snapshot decode error in
src/kvraft/server.go")
        return
    }
    if index != -1 && index != lastIncludedIndex {
        utils.Debugger(utils.DWarn, "server snapshot doesn't match
m.SnapshotIndex in src/kvraft/server.go")
        return
    }

    kv.mu.Lock()
    kv.database = serviceLayerState[0].(map[string]string)
    kv.duplicate = serviceLayerState[1].(map[int64]Result)
    kv.mu.Unlock()
}

```

```

func (kv *KVServer) loadSnapshot(snapshot []byte) {
    if snapshot == nil || len(snapshot) < 1 { return }

    buffer := bytes.NewBuffer(snapshot)
    decoder := labgob.NewDecoder(buffer)
    var lastIncludedIndex int
    var serviceLayerState []interface{}
    var lastIncludedTerm int
    if decoder.Decode(&lastIncludedIndex) != nil ||
decoder.Decode(&serviceLayerState) != nil ||
    decoder.Decode(&lastIncludedTerm) != nil {
        utils.Debugger(utils.DError, "Can't decode snapshot!")
        return
    }

    kv.mu.Lock()
    kv.database = serviceLayerState[0].(map[string]string)
    kv.duplicate = serviceLayerState[1].(map[int64]Result)
    kv.mu.Unlock()
}

```